OpenLDAP 2.6 sous Docker
Manuel d'exploitation et de validation

Thomas Broine

Résumé

Guide pas à pas pour monter OpenLDAP sous Docker, avec **commandes et LDIF à copier-coller**. Les chapitres techniques suivent une même logique : **objectif**, **notions**, **déroulé** (étapes ou paramètres), lien avec l'**annexe** lorsqu'un script reprend la même chose, puis **tests** et résultats attendus. L'**annexe** regroupe les fichiers Docker et les scripts du build ; le détail LDAP manuel est dans le corps du document (sauf Keycloak, dont le script complet est en annexe).

Table des matières

1 Introduction : annuaire LDAP et périmètre du lab **3**
1.1 Objets et vocabulaire . . . 3
1.2 Architecture logique du lab . . . 3
1.3 Organisation des chapitres suivants . . . 3

2 Comment utiliser ce guide **4**

3 Prérequis et démarrage de l'infrastructure **5**
3.1 Objectif de ce chapitre . . . 5
3.2 Ce que vous manipulez . . . 5
3.3 Logiciels . . . 5
3.4 Ports par défaut . . . 5
3.5 Identifiants de démonstration . . . 5
3.6 Démarrage de la stack . . . 6
3.7 Contrôle rapide . . . 6

4 Initialisation OpenLDAP : fichiers LDIF et commandes (pas à pas) **7**
4.1 Objectif de ce chapitre . . . 7
4.2 Deux espaces de configuration . . . 7
4.3 Déroulé résumé (avant le détail pas à pas) . . . 7
4.4 Étape 1 - Explorer `cn=config` . . . 8
4.5 Étape 2 - (Optionnel) Niveau de trace sur les ACL . . . 8
4.6 Étape 3 - Retirer la base MDB par défaut (`del_db.ldif`) . . . 8
4.7 Étape 4 - Créer la MDB cible (`add_suffix.ldif`) . . . 8
4.8 Étape 5 - Constat : le suffixe existe dans `cn=config`, pas encore dans les données 9
4.9 Étape 6 - Peupler : racine, OU, groupes, utilisateurs (`fill_db.ldif`) . . . 9
4.10 Étape 7 - Remplacer les membres factices des groupes . . . 10
4.11 Étape 8 - ACL sur `olcDatabase={1}mdb,cn=config` . . . 11
4.12 Contrôles finaux sur le DIT . . . 12

5 Rôles fonctionnels et validation des ACL **13**
5.1 Objectif de ce chapitre . . . 13
5.2 Idée des règles du lab . . . 13
5.3 Rôles retenus . . . 13
5.4 Test : droits contrastés (comportement attendu) . . . 13
5.5 Fiche de test - authentification et appartenance . . . 14

6 Intégration Linux : attributs POSIX, NSS et PAM **15**
6.1 Objectif de ce chapitre . . . 15
6.2 NSS, PAM et nslcd en bref . . . 15
6.3 Déroulé des opérations . . . 15
6.4 LDIF : attributs POSIX sur les utilisateurs . . . 15

6.5 LDIF : groupes `posixGroup` . . . 16
6.6 Fichiers NSS, nslcd et PAM dans le conteneur `ldap` . . . 17
6.7 Fiche de test - NSS dans le conteneur `ldap` . . . 19
6.8 Authentification (`su`) . . . 19

7 Keycloak : fédération LDAP **20**
7.1 À quoi sert Keycloak ici ? . . . 20
7.2 User Federation LDAP . . . 20
7.3 Console, ports et réseau . . . 20
7.4 Paramètres LDAP du lab . . . 20
7.5 Variables d'environnement du script (annexe A.10) . . . 21
7.6 Déroulement du script (API d'administration REST) . . . 21
7.7 Exécution : copier le script depuis l'annexe . . . 21
7.8 Vérification - requêtes `curl` (aperçu) . . . 21

8 Réplication : fournisseur / réplica **23**
8.1 Objectif de ce chapitre . . . 23
8.2 Notions . . . 23
8.3 Fournisseur : module `syncprov`, overlay, `olcServerID`, index . . . 23
8.4 Réplica : `olcServerID`, lecture seule, `syncrepl` . . . 24
8.5 Fiche de test - propagation et interdiction d'écriture . . . 24

9 Méta-annuaire (back-meta) **26**
9.1 Objectif de ce chapitre . . . 26
9.2 Vue d'ensemble . . . 26
9.3 Déroulé logique de la configuration . . . 26
9.4 Contrôle des contextes . . . 27
9.5 Fiche de test - deux sources pour `uid=thomas` . . . 27

A Sources complètes du laboratoire **29**
A.1 Vue d'ensemble des fichiers . . . 29
A.2 `docker-compose.yml` . . . 29
A.3 `docker/Dockerfile` . . . 32
A.4 `docker/entrypoint.sh` . . . 32
A.5 Script `scripts/init_ldap.sh` . . . 34
A.6 Script `scripts/init_ldap_linux_integration.sh` . . . 38
A.7 Script `scripts/init_replication_provider.sh` . . . 42
A.8 Script `scripts/init_replication_consumer.sh` . . . 43
A.9 Script `scripts/init_meta_annuaire.sh` . . . 44
A.10 Script `scripts/configure_keycloak_ldap.sh` (machine hôte) . . . 46

Chapitre 1

Introduction : annuaire LDAP et périmètre du lab

1.1 Objets et vocabulaire

Un annuaire LDAP stocke des **entrées** identifiées par un **DN** (nom distinctif), organisées en arbre sous un **suffixe** (base DN). Les schémas fixent les **classes d'objet** et **attributs** autorisés. OpenLDAP 2.6 sépare couramment :

— `cn=config` : configuration du serveur (bases, modules, ACL `olcAccess`, réplication) ;

— le **suffixe métier** (ex. `dc=example,dc=org`) : utilisateurs, groupes, applications.

1.2 Architecture logique du lab

```
Hôte (Docker)
  ldap:389          -> fournisseur RW, DIT dc=example,dc=org (+ syncprov)
  ldap-replica:1389 -> réplica RO (syncrepl)
  ldap-acme:2389    -> second annuaire dc=acme,dc=com
  ldap-meta:3389    -> méta back-meta, suffixe o=federation
  keycloak:8090     -> Keycloak (fédération LDAP -> realm tp-ldap)
```

L'annexe A contient les fichiers pour lancer la stack Docker du TP. La suite du document décrit surtout les **commandes LDAP** équivalentes à une mise en place manuelle.

1.3 Organisation des chapitres suivants

Partie	**Contenu**
Prérequis	Stack Docker, identifiants, premier contrôle LDAP.
Init LDIF	Création du DIT et des ACL (cœur du serveur `ldap`).
Rôles / ACL	Validation des droits et des authentifications.
Linux	NSS/PAM et comptes POSIX dans le conteneur.
Keycloak	Fédération LDAP, API admin, script en annexe.
Réplication	Fournisseur / réplica et tests de propagation.
Méta	Agrégation `ldap` + `ldap-acme` sous `o=federation`.
Synthèse	Enchaînement global et dépannage.

Chapitre 2

Comment utiliser ce guide

Le fil principal est : **commandes et LDIF** dans chaque chapitre. Pour monter les conteneurs, reprenez le compose et l'image depuis l'annexe (A.2 à A.4).

Ordre conseillé : (1) fichiers Docker depuis l'annexe ; (2) `docker compose up -d --build` (ch. 3) ; (3) ch. 4, puis intégration Linux, ACL, réplication, méta, Keycloak.

Remarque : Dans les listings, `\{1\}` sert à LaTeX ; dans vos LDIF réels, mettez `{1}`.

Remarque : Forme des chapitres techniques : comme pour Keycloak, chaque grande partie commence par l'**objectif** et des **notions utiles**, puis un **déroulé** (étapes numérotées ou commandes), le lien avec l'**annexe** lorsqu'un script reprend la même logique, et enfin des **tests** avec **résultat attendu**.

Chapitre 3

Prérequis et démarrage de l'infrastructure

3.1 Objectif de ce chapitre

Mettre en route la **stack Docker** du TP et vérifier que le service LDAP principal répond avant d'appliquer les LDIF des chapitres suivants (ou de vous appuyer sur l'automatisation déjà embarquée dans l'image).

3.2 Ce que vous manipulez

— **Docker Compose** : décrit les services (`ldap`, réplicas, méta, Keycloak), les ports publiés sur l'hôte et les variables d'environnement. Fichier complet : annexe A.2.

— **Image LDAP** : construction via `Dockerfile` et démarrage via `entrypoint.sh` (annexes A.3 et A.4) : paquets OpenLDAP, scripts copiés dans `/container/init.d/`, exécution dans un ordre fixe puis fichier sentinelle `/container/.ldap-init-complete` pour les `healthcheck`.

3.3 Logiciels

— Docker et plugin `docker compose` ;

— sur l'hôte : `ldap-utils` (Debian/Ubuntu) ou clients LDAP équivalents ;

— pour Keycloak en ligne de commande : `curl`, `python3`.

3.4 Ports par défaut

`389`, `1389`, `2389`, `3389`, `8090`, `9001`. En cas de conflit, changez les ports dans `docker-compose.yml` (annexe A.2).

3.5 Identifiants de démonstration

Usage	Valeur
RootDN annuaires `example` / `acme`	`cn=admin,<suffixe>` / `admin`
Méta-annuaire	`cn=admin,o=federation` / `admin`
Utilisateur	`uid=thomas` / `thomas123`
Utilisateur	`uid=john` / `john123`
Admin Keycloak	`admin` / `admin`

3.6 Démarrage de la stack

Prérequis : avoir recréé l'arborescence du projet ou cloné le dépôt ; le répertoire de travail pour la commande ci-dessous est celui qui contient `docker-compose.yml` (annexe A.2).

```
cd chemin/vers/TP-LDAP/projet
docker compose up -d --build
docker compose ps
```

3.7 Contrôle rapide

```
docker ps --format '{{.Names}}' | grep -qx ldap && echo "conteneur ldap OK"
ldapsearch -x -H ldap://localhost:389 -b dc=example,dc=org -s base dn
ldapwhoami -x -D cn=admin,dc=example,dc=org -w admin
```

Résultat attendu : `conteneur ldap OK` ; `ldapsearch` montre `dn: dc=example,dc=org` ; `ldapwhoami` montre le RootDN. Si besoin, attendre que `docker compose ps` affiche `healthy` pour `ldap`.

Chapitre 4

Initialisation OpenLDAP : fichiers LDIF et commandes (pas à pas)

4.1 Objectif de ce chapitre

Obtenir un annuaire `dc=example,dc=org` avec racine, unités `ou=people` / `ou=groups`, utilisateurs de démo, groupes et **ACL** alignées sur le modèle du TP - en rejouant les opérations **à la main** (fichiers LDIF + commandes). Cela correspond à ce que fait le script `init_ldap.sh` dans l'image (annexe A.5) au premier démarrage du conteneur `ldap`.

4.2 Deux espaces de configuration

`cn=config` : base de configuration OpenLDAP (définition des bases MDB, modules, ACL `olcAccess`). Les modifications passent par le mécanisme **SASL EXTERNAL** sur le socket **ldapi** depuis le conteneur (`docker exec ldap ldapmodify -Y EXTERNAL -H ldapi:// ...`) : pas de mot de passe RootDN, il faut être root local au sens du conteneur.

Suffixe métier (`dc=example,dc=org`) : les données « métier » (personnes, groupes). Ici on utilise une **liaison simple** (`-x`) avec le RootDN défini dans `cn=config`, en ciblant `ldap://localhost:389` depuis l'hôte (port publié par Docker).

On prépare des **fichiers LDIF**, puis `ldapadd` / `ldapmodify` les appliquent.

Remarque : Si une entrée existe déjà, OpenLDAP renvoie *already exists* : c'est normal. Base vide : `docker compose down -v` puis `up -d --build`. Après avoir retiré une MDB dans `cn=config`, pensez à vider `/var/lib/ldap` et à redémarrer `slapd`.

4.3 Déroulé résumé (avant le détail pas à pas)

1. Lire `cn=config` et repérer les MDB (étape 1).
2. (Optionnel) Activer les logs ACL (étape 2).
3. Supprimer la MDB par défaut ; en manuel purger `/var/lib/ldap` et relancer `slapd` si besoin (étape 3).
4. Créer la MDB pour `dc=example,dc=org` avec hash du mot de passe admin (étape 4).
5. Vérifier que les données du suffixe sont encore absentes (étape 5).
6. Peupler le DIT et corriger les membres des groupes (étapes 6–7).
7. Poser les `olcAccess` en deux temps (étape 8).

4.4 Étape 1 - Explorer `cn=config`

`cn=config` est la config du serveur. `-Y EXTERNAL -H ldapi:///` = auth locale (root sur la machine), sans mot de passe. La première commande lit la racine `cn=config` ; la seconde liste les bases MDB (`olcSuffix`, `olcRootDN`).

```
# Meme idee que : ldapsearch -Y EXTERNAL -H ldapi:/// -b "cn=config"
docker exec ldap ldapsearch -Y EXTERNAL -H ldapi:/// -b cn=config -s base dn
docker exec ldap ldapsearch -Y EXTERNAL -H ldapi:/// \
  -b cn=config -s sub "(olcDatabase=mdb)" dn olcSuffix olcRootDN
```

4.5 Étape 2 - (Optionnel) Niveau de trace sur les ACL

Le fichier modifie l'entrée `cn=config` pour fixer `olcLogLevel` à `acl`. OpenLDAP journalise alors les décisions d'accès (qui a été autorisé ou refusé, sur quelle règle), ce qui aide à déboguer des problèmes d'`olcAccess`. Le `replace` remplace entièrement la valeur précédente du niveau de log pour cet attribut.

```
cat > /tmp/log_acl.ldif <<'EOF'
dn: cn=config
changetype: modify
replace: olcLogLevel
olcLogLevel: acl
EOF
docker cp /tmp/log_acl.ldif ldap:/tmp/log_acl.ldif
docker exec ldap ldapmodify -Y EXTERNAL -H ldapi:/// -f /tmp/log_acl.ldif
# Puis sur l'hote : docker logs -f ldap   (ou journalctl -u slapd sur Debian nue)
```

Pour revenir au calme : remplacer `olcLogLevel: acl` par `none` dans le même LDIF (`replace`).

4.6 Étape 3 - Retirer la base MDB par défaut (`del_db.ldif`)

L'installation par défaut crée souvent une base `mdb` numérotée `{1}` avec un suffixe générique (`dc=nodomain`, etc.). Ce LDIF **supprime** cette entrée de configuration : on ne supprime pas encore les fichiers sur disque, mais on retire la base telle qu'OpenLDAP la connaît dans `cn=config`. C'est la même idée que « retirer l'ancien contexte » avant d'en déclarer un nouveau. Le numéro `{1}` est l'index interne de la première base MDB ; il peut varier si d'autres bases ont été ajoutées (vérifier à l'étape 1).

```
cat > /tmp/del_db.ldif <<'EOF'
dn: olcDatabase=\{1\}mdb,cn=config
changetype: delete
EOF
docker cp /tmp/del_db.ldif ldap:/tmp/del_db.ldif
docker exec ldap ldapmodify -Y EXTERNAL -H ldapi:/// -f /tmp/del_db.ldif
```

En manuel, après cette étape : vider `/var/lib/ldap` et relancer `slapd` si le suffixe sur disque ne correspond plus à la config.

4.7 Étape 4 - Créer la MDB cible (`add_suffix.ldif`)

On **ajoute** une entrée `olcDatabase=mdb,cn=config` (souvent numérotée `{1}`). Champs utiles : `olcSuffix`, `olcRootDN`, `olcRootPW`, `olcDbDirectory`, `olcDbIndex`. Ici : `ldapadd` car nouvelle entrée dans `cn=config`.

Générer le hash du mot de passe administrateur (ex. `admin`) :

```
docker exec ldap slappasswd -s admin
```

Coller la valeur `{SSHA}...` à la place de `ADMIN_HASH` ci-dessous.

```
cat > /tmp/add_suffix.ldif <<'EOF'
dn: olcDatabase=mdb,cn=config
objectClass: olcDatabaseConfig
objectClass: olcMdbConfig
olcDatabase: mdb
olcSuffix: dc=example,dc=org
olcRootDN: cn=admin,dc=example,dc=org
olcRootPW: ADMIN_HASH
olcDbDirectory: /var/lib/ldap
olcDbIndex: objectClass eq
olcDbIndex: cn,sn,uid eq
olcDbMaxSize: 1073741824
EOF
docker cp /tmp/add_suffix.ldif ldap:/tmp/add_suffix.ldif
docker exec ldap ldapadd -Y EXTERNAL -H ldapi:/// -f /tmp/add_suffix.ldif
```

4.8 Étape 5 - Constat : le suffixe existe dans `cn=config`, pas encore dans les données

Recherche sur le **DIT données** (pas `cn=config`) : sans racine `dc=example,dc=org`, réponse `No such object` (normal tant que rien n'a été ajouté avec `ldapadd`).

```
docker exec ldap ldapsearch -Y EXTERNAL -H ldapi:/// -b "dc=example,dc=org" -s base dn
```

Résultat attendu : `result: 32 No such object`.

4.9 Étape 6 - Peupler : racine, OU, groupes, utilisateurs (`fill_db.ldif`)

Un seul LDIF : racine, `ou=people`, `ou=groups`, trois `groupOfNames` avec membre factice `cn=dummy` (retiré à l'étape 7), deux `inetOrgPerson` avec mots de passe **hashés**. `ldapadd -x` avec le RootDN (étape 4) vers `localhost:389`.

Hashes pour `THOMAS_HASH` / `JOHN_HASH` :

```
docker exec ldap slappasswd -s thomas123
docker exec ldap slappasswd -s john123
```

```
cat > /tmp/fill_db.ldif <<'EOF'
dn: dc=example,dc=org
objectClass: top
objectClass: dcObject
objectClass: organization
o: Example Org
dc: example

dn: ou=people,dc=example,dc=org
objectClass: top
objectClass: organizationalUnit
ou: people

dn: ou=groups,dc=example,dc=org
objectClass: top
```

```
objectClass: organizationalUnit
ou: groups

dn: cn=admin_ldap,ou=groups,dc=example,dc=org
objectClass: top
objectClass: groupOfNames
cn: admin_ldap
member: cn=dummy,dc=example,dc=org

dn: cn=developers,ou=groups,dc=example,dc=org
objectClass: top
objectClass: groupOfNames
cn: developers
member: cn=dummy,dc=example,dc=org

dn: cn=admin_keycloak,ou=groups,dc=example,dc=org
objectClass: top
objectClass: groupOfNames
cn: admin_keycloak
member: cn=dummy,dc=example,dc=org

dn: uid=thomas,ou=people,dc=example,dc=org
objectClass: top
objectClass: inetOrgPerson
cn: Thomas
sn: Thomas
uid: thomas
mail: thomas@example.org
userPassword: THOMAS_HASH

dn: uid=john,ou=people,dc=example,dc=org
objectClass: top
objectClass: inetOrgPerson
cn: John
sn: John
uid: john
mail: john@example.org
userPassword: JOHN_HASH
EOF
ldapadd -x -H ldap://localhost:389 -D cn=admin,dc=example,dc=org -w admin -f
    /tmp/fill_db.ldif
```

Remarque : Avant `docker cp` / `ldapadd`, ouvrir `/tmp/add_suffix.ldif` et `/tmp/fill_db.ldif` dans un éditeur pour remplacer `ADMIN_HASH`, `THOMAS_HASH` et `JOHN_HASH` par les sorties de `slappasswd`.

4.10 Étape 7 - Remplacer les membres factices des groupes

Trois modifications sur les groupes : `delete: member` (dummy) puis `add: member` (vrai utilisateur), séparées par `-` dans le même bloc `modify`.

```
cat > /tmp/group_members.ldif <<'EOF'
dn: cn=admin_ldap,ou=groups,dc=example,dc=org
changetype: modify
delete: member
member: cn=dummy,dc=example,dc=org
-
```

```
add: member
member: uid=thomas,ou=people,dc=example,dc=org

dn: cn=developers,ou=groups,dc=example,dc=org
changetype: modify
delete: member
member: cn=dummy,dc=example,dc=org
-
add: member
member: uid=john,ou=people,dc=example,dc=org

dn: cn=admin_keycloak,ou=groups,dc=example,dc=org
changetype: modify
delete: member
member: cn=dummy,dc=example,dc=org
-
add: member
member: uid=thomas,ou=people,dc=example,dc=org
EOF
ldapmodify -x -H ldap://localhost:389 -D cn=admin,dc=example,dc=org -w admin -f
    /tmp/group_members.ldif
```

4.11 Étape 8 - ACL sur `olcDatabase={1}mdb,cn=config`

Les ACL sont des lignes `olcAccess` sur l'entrée de la MDB dans `cn=config`. Syntaxe : `to ... by ... read|write|manage|auth|none`. L'**ordre** des règles compte : la première qui correspond s'applique.

Deux fichiers : d'abord une règle large (RootDN `manage`, autres `read`), puis trois règles (mots de passe ; racine vide du DIT en lecture ; délégation au groupe `admin_ldap` et lecture pour les utilisateurs authentifiés).

```
cat > /tmp/acl_base.ldif <<'EOF'
dn: olcDatabase=\{1\}mdb,cn=config
changetype: modify
add: olcAccess
olcAccess: to * by dn.exact=cn=admin,dc=example,dc=org manage by * read
EOF
docker cp /tmp/acl_base.ldif ldap:/tmp/acl_base.ldif
docker exec ldap ldapmodify -Y EXTERNAL -H ldapi:/// -f /tmp/acl_base.ldif

cat > /tmp/acl_deleg.ldif <<'EOF'
dn: olcDatabase=\{1\}mdb,cn=config
changetype: modify
add: olcAccess
olcAccess: to attrs=userPassword by
    group.exact=cn=admin_ldap,ou=groups,dc=example,dc=org write by self write by
    anonymous auth by * none
olcAccess: to dn.base="" by * read
olcAccess: to * by group.exact=cn=admin_ldap,ou=groups,dc=example,dc=org manage by
    self write by users read by * none
EOF
docker cp /tmp/acl_deleg.ldif ldap:/tmp/acl_deleg.ldif
docker exec ldap ldapmodify -Y EXTERNAL -H ldapi:/// -f /tmp/acl_deleg.ldif
```

Remarque : Pour voir l'ordre réel des règles : `docker exec ldap ldapsearch -Y EXTERNAL -H ldapi:/// -b olcDatabase={1}mdb,cn=config -s base olcAccess`.

4.12 Contrôles finaux sur le DIT

Vérifier les personnes sous `ou=people`, le mail de `thomas`, et les `member` du groupe `admin_ldap`.

```
ldapsearch -x -H ldap://localhost:389 -b ou=people,dc=example,dc=org -s one dn
ldapsearch -x -H ldap://localhost:389 -b ou=people,dc=example,dc=org "(uid=thomas)"
    dn mail
ldapsearch -x -H ldap://localhost:389 -b ou=groups,dc=example,dc=org
    "(cn=admin_ldap)" member
```

Chapitre 5

Rôles fonctionnels et validation des ACL

5.1 Objectif de ce chapitre

Montrer **à l'usage** ce que font les ACL posées à l'étape 8 du chapitre 4 : qui peut créer ou modifier des entrées, qui ne peut pas, et comment vérifier les liaisons et les appartenances aux groupes.

5.2 Idée des règles du lab

Le RootDN `cn=admin,...` reste un compte de secours avec tous les droits sur les données. Le groupe `admin_ldap` reçoit une **délégation** : ses membres peuvent gérer le contenu du suffixe selon les `olcAccess` (gestion des comptes, etc.). Les utilisateurs standards ont surtout le droit de lire et de modifier leur propre entrée ; le mot de passe est traité par des règles dédiées (`userPassword`, `auth` pour les liaisons).

5.3 Rôles retenus

Groupe	Rôle opérationnel
`admin_ldap`	Administration déléguée du suffixe (hors usage courant du seul RootDN)
`developers`	Groupe fonctionnel exemple
`admin_keycloak`	Gestion procédurale Keycloak côté organisation

Le RootDN `cn=admin,...` reste le filet de secours serveur ; la délégation opérationnelle passe par `admin_ldap` (voir étape 8 du chapitre 4).

5.4 Test : droits contrastés (comportement attendu)

On tente d'ajouter un utilisateur `testacl` avec le **même** fichier LDIF : d'abord bind `john` (pas dans `admin_ldap`) - échec ; puis bind `thomas` - succès ; enfin suppression par `thomas`. Le `|| true` évite d'arrêter le script sur l'échec voulu de la première commande.

```
docker exec ldap slappasswd -s testpass123
cat > /tmp/user-testacl.ldif <<'EOF'
dn: uid=testacl,ou=people,dc=example,dc=org
objectClass: top
objectClass: inetOrgPerson
cn: Test
sn: ACL
uid: testacl
userPassword: REMPLACER_PAR_HASH_SSHA
```

```
EOF
# Echec attendu (john)
ldapadd -x -H ldap://localhost:389 \
  -D uid=john,ou=people,dc=example,dc=org -w john123 -f /tmp/user-testacl.ldif || true
# Succes attendu (thomas)
ldapadd -x -H ldap://localhost:389 \
  -D uid=thomas,ou=people,dc=example,dc=org -w thomas123 -f /tmp/user-testacl.ldif
ldapdelete -x -H ldap://localhost:389 \
  -D uid=thomas,ou=people,dc=example,dc=org -w thomas123
    uid=testacl,ou=people,dc=example,dc=org
```

5.5 Fiche de test - authentification et appartenance

Résultats à interpréter : `ldapwhoami` doit renvoyer le DN pour chaque compte si le mot de passe est bon. Les recherches sur `admin_ldap` et `developers` affichent les `member` (DN) tant que les groupes sont en `groupOfNames`. Les deux dernières commandes vérifient qu'un utilisateur authentifié peut lire la base du suffixe (`-s base`), cohérent avec `by users read` dans les ACL.

```
ldapwhoami -x -H ldap://localhost:389 -D cn=admin,dc=example,dc=org -w admin
ldapwhoami -x -H ldap://localhost:389 -D uid=thomas,ou=people,dc=example,dc=org -w
    thomas123
ldapwhoami -x -H ldap://localhost:389 -D uid=john,ou=people,dc=example,dc=org -w
    john123
ldapsearch -x -H ldap://localhost:389 -b ou=groups,dc=example,dc=org
    "(cn=admin_ldap)" member memberUid
ldapsearch -x -H ldap://localhost:389 -b ou=groups,dc=example,dc=org
    "(cn=developers)" member memberUid
ldapsearch -x -H ldap://localhost:389 -D uid=thomas,ou=people,dc=example,dc=org -w
    thomas123 \
  -b dc=example,dc=org -s base dn
ldapsearch -x -H ldap://localhost:389 -D uid=john,ou=people,dc=example,dc=org -w
    john123 \
  -b dc=example,dc=org -s base dn
```

Chapitre 6

Intégration Linux : attributs POSIX, NSS et PAM

6.1 Objectif de ce chapitre

Que le conteneur `ldap` se comporte comme une machine Linux qui **résout** les utilisateurs/-groupes via l'annuaire (`getent`, `id`) et peut ouvrir une session (`su`) avec mot de passe LDAP. Même logique que le script `init_ldap_linux_integration.sh` (annexe A.6), exécuté sur le fournisseur après `init_ldap.sh`.

6.2 NSS, PAM et nslcd en bref

— **NSS** (`nsswitch.conf`) : indique où la libc cherche passwd/group/shadow (fichiers locaux puis LDAP).

— **nslcd** : démon qui répond aux requêtes NSS en interrogeant LDAP selon `/etc/nslcd.conf` (URI, base, filtres sur `posixAccount` / `posixGroup`).

— **PAM** : chaînes `common-auth`, `common-account`, etc., avec `pam_ldap.so` pour accepter une auth contre LDAP ; `pam_mkhomedir` crée le home à la première session.

Les entrées `inetOrgPerson` du DIT ne suffisent pas : il faut ajouter les classes **POSIX** (UID, GID, shell, home) et des groupes **posixGroup** avec `memberUid`, ce qui implique souvent de remplacer les anciens `groupOfNames`.

6.3 Déroulé des opérations

1. **LDIF utilisateurs** : ajouter `posixAccount` / `shadowAccount` et les attributs numériques (section suivante).
2. **LDIF groupes** : supprimer si besoin les `groupOfNames`, recréer des `posixGroup` avec `memberUid`.
3. **Fichiers système** : `nsswitch.conf`, `ldap.conf`, `nslcd.conf`, `/etc/ldap.conf` (PAM), puis les quatre `/etc/pam.d/common-*`.
4. **Redémarrage** : `nslcd`, `nscd`, invalidation du cache.
5. **Tests** : `getent`, puis `su`.

6.4 LDIF : attributs POSIX sur les utilisateurs

Chaque bloc `changetype: modify` regroupe plusieurs `add:` sur **une** entrée (séparés par `-`) : une seule modification atomique par utilisateur.

```
cat > /tmp/posix_users.ldif <<'EOF'
dn: uid=thomas,ou=people,dc=example,dc=org
changetype: modify
add: objectClass
objectClass: posixAccount
objectClass: shadowAccount
-
add: uidNumber
uidNumber: 1001
-
add: gidNumber
gidNumber: 1001
-
add: homeDirectory
homeDirectory: /home/thomas
-
add: loginShell
loginShell: /bin/bash
-
add: gecos
gecos: Thomas Thomas

dn: uid=john,ou=people,dc=example,dc=org
changetype: modify
add: objectClass
objectClass: posixAccount
objectClass: shadowAccount
-
add: uidNumber
uidNumber: 1002
-
add: gidNumber
gidNumber: 1002
-
add: homeDirectory
homeDirectory: /home/john
-
add: loginShell
loginShell: /bin/bash
-
add: gecos
gecos: John John
EOF
ldapmodify -x -H ldap://localhost:389 -D cn=admin,dc=example,dc=org -w admin -f
    /tmp/posix_users.ldif
```

6.5 LDIF : groupes `posixGroup`

NSS attend des groupes avec un `gidNumber` numérique et des `memberUid` (login Unix), pas des `member` en DN complets comme `groupOfNames`. On supprime donc souvent les anciennes entrées `groupOfNames` puis on recrée des `posixGroup` avec les mêmes noms logiques (`cn=admin_ldap`, etc.).

Après suppression des entrées `groupOfNames` existantes si besoin (`ldapdelete`), recréer :

```
cat > /tmp/posix_groups.ldif <<'EOF'
dn: cn=admin_ldap,ou=groups,dc=example,dc=org
```

```
objectClass: top
objectClass: posixGroup
cn: admin_ldap
gidNumber: 1001
memberUid: thomas

dn: cn=developers,ou=groups,dc=example,dc=org
objectClass: top
objectClass: posixGroup
cn: developers
gidNumber: 1002
memberUid: john
EOF
ldapadd -x -H ldap://localhost:389 -D cn=admin,dc=example,dc=org -w admin -f
    /tmp/posix_groups.ldif
```

6.6 Fichiers NSS, nslcd et PAM dans le conteneur `ldap`

`nsswitch.conf` : fichiers puis LDAP. `nslcd.conf` : URI, base, bind admin, filtres POSIX. PAM : `pam_ldap.so` dans les `common-*`. Puis redémarrer `nslcd` et vider le cache `nscd` si besoin.

```
cat > /tmp/nsswitch.conf <<'EOF'
passwd:         files ldap
group:          files ldap
shadow:         files ldap
gshadow:        files
hosts:          files dns
networks:       files
protocols:      db files
services:       db files
ethers:         db files
rpc:            db files
netgroup:       ldap
EOF
docker cp /tmp/nsswitch.conf ldap:/etc/nsswitch.conf

cat > /tmp/ldap_ldap.conf <<'EOF'
BASE dc=example,dc=org
URI ldap://localhost:389
TLS_REQCERT never
EOF
docker cp /tmp/ldap_ldap.conf ldap:/etc/ldap/ldap.conf

cat > /tmp/nslcd.conf <<'EOF'
uri ldap://localhost:389
base dc=example,dc=org
ldap_version 3
binddn cn=admin,dc=example,dc=org
bindpw admin
filter passwd (objectClass=posixAccount)
filter group (objectClass=posixGroup)
filter shadow (objectClass=shadowAccount)
map passwd homeDirectory homeDirectory
map passwd uidNumber uidNumber
map passwd gidNumber gidNumber
map passwd loginShell loginShell
map passwd gecos gecos
```

```
map group gidNumber gidNumber
map group memberUid memberUid
EOF
docker cp /tmp/nslcd.conf ldap:/etc/nslcd.conf
docker exec ldap chmod 600 /etc/nslcd.conf

cat > /tmp/ldap.conf <<'EOF'
base dc=example,dc=org
uri ldap://localhost:389
ldap_version 3
rootbinddn cn=admin,dc=example,dc=org
binddn cn=admin,dc=example,dc=org
bindpw admin
pam_password crypt
nss_base_passwd ou=people,dc=example,dc=org
nss_base_shadow ou=people,dc=example,dc=org
nss_base_group ou=groups,dc=example,dc=org
pam_login_attribute uid
pam_member_attribute member
pam_filter objectclass=inetOrgPerson
EOF
docker cp /tmp/ldap.conf ldap:/etc/ldap.conf

cat > /tmp/common-auth <<'EOF'
auth    sufficient      pam_ldap.so
auth    sufficient      pam_unix.so nullok_secure use_first_pass
auth    required        pam_deny.so
EOF
docker cp /tmp/common-auth ldap:/etc/pam.d/common-auth

cat > /tmp/common-account <<'EOF'
account sufficient      pam_ldap.so
account sufficient      pam_unix.so
account required        pam_deny.so
EOF
docker cp /tmp/common-account ldap:/etc/pam.d/common-account

cat > /tmp/common-password <<'EOF'
password        sufficient      pam_ldap.so
password        sufficient      pam_unix.so nullok obscure min=4 max=8 md5
password        required        pam_deny.so
EOF
docker cp /tmp/common-password ldap:/etc/pam.d/common-password

cat > /tmp/common-session <<'EOF'
session required        pam_mkhomedir.so skel=/etc/skel umask=0022
session sufficient      pam_ldap.so
session sufficient      pam_unix.so
session required        pam_deny.so
EOF
docker cp /tmp/common-session ldap:/etc/pam.d/common-session

docker exec ldap service nslcd restart || docker exec ldap service nslcd start
docker exec ldap service nscd restart || docker exec ldap service nscd start
docker exec ldap nscd -i passwd 2>/dev/null || true
docker exec ldap nscd -i group 2>/dev/null || true
```

6.7 Fiche de test - NSS dans le conteneur `ldap`

```
docker exec ldap getent passwd thomas
docker exec ldap getent passwd john
docker exec ldap getent group admin_ldap
docker exec ldap getent group developers
```

Résultat attendu : lignes `passwd` et `group` contenant les UID/GID définis dans le LDIF.

6.8 Authentification (`su`)

```
docker exec -it ldap su - thomas -c 'id'
```

Résultat attendu : sortie du type `uid=1001(thomas) gid=1001(thomas) groups=1001(thomas)` (les numéros correspondent au LDIF POSIX).

La création du répertoire personnel dépend de `pam_mkhomedir.so` dans `common-session` (bloc ci-dessus).

Chapitre 7

Keycloak : fédération LDAP

7.1 À quoi sert Keycloak ici ?

Keycloak est un serveur d'identité (gestion des comptes et des protocoles du type OpenID Connect / OAuth2). Dans ce TP il ne remplace pas LDAP : il sert de **point d'entrée** pour des applications qui parleraient à Keycloak, pendant que les comptes « source » restent dans OpenLDAP.

Un **realm** est un espace isolé (utilisateurs, clients, paramètres). Le realm **master** est celui de l'administration Keycloak. Le realm applicatif du lab est `tp-ldap` : c'est là que nous branchons LDAP.

7.2 User Federation LDAP

La **fédération utilisateur** (User Federation) indique à Keycloak comment joindre l'annuaire : URL, bind, base des utilisateurs, attribut de login (`uid`), etc. Keycloak importe alors une **copie de travail** des utilisateurs (attributs utiles au login). Ici le mode est `READ_ONLY` : le mot de passe reste contrôlé côté LDAP. L'appel `triggerFullSync` demande une synchronisation complète après création du connecteur.

7.3 Console, ports et réseau

— Console d'administration : `http://localhost:8090/admin/` - compte bootstrap `admin`/`admin` (défini dans le compose, annexe A.2).

— Depuis le **conteneur** Keycloak, l'hôte `localhost` n'est pas votre PC : le service LDAP s'appelle `ldap` sur le réseau Docker, d'où l'URI `ldap://ldap:389` dans la configuration.

7.4 Paramètres LDAP du lab

Ces valeurs sont celles injectées dans le JSON du fournisseur (voir annexe A.10) :

Champ (Keycloak)	**Valeur**
Connection URL	`ldap://ldap:389`
Users DN	`ou=people,dc=example,dc=org`
Bind DN	`cn=admin,dc=example,dc=org`
Bind credential	mot de passe RootDN LDAP (`admin` au lab)
Username LDAP attribute	`uid`
UUID LDAP attribute	`entryUUID`
User object classes	`inetOrgPerson`, `organizationalPerson`

7.5 Variables d'environnement du script (annexe A.10)

Le script lit des variables optionnelles (surcharge des défauts) :

Variable	Rôle (défaut)
`KEYCLOAK_URL`	URL HTTP de Keycloak (`http://localhost:8090`).
`KEYCLOAK_ADMIN` / `KEYCLOAK_ADMIN_PASSWORD`	Compte admin console (`admin`/`admin`).
`KEYCLOAK_REALM`	Nom du realm applicatif (`tp-ldap`).
`KEYCLOAK_LDAP_CONNECTION`	URI vue depuis le conteneur Keycloak (`ldap://ldap:389`
`LDAP_BIND_DN`, `LDAP_BIND_PW`, `LDAP_USERS_DN`	Connexion lecture à l'annuaire.

7.6 Déroulement du script (API d'administration REST)

Tout le code exécutable est dans l'annexe A.10. Voici la logique, dans l'ordre :

1. **Attendre Keycloak** : requête sur `/realms/master` jusqu'à réponse OK.
2. **Obtenir un jeton** `access_token` : `POST` sur `/realms/master/protocol/openid-connect/token` avec `grant_type=password`, client `admin-cli`, identifiants admin. C'est le flux OAuth2 « mot de passe du propriétaire de la ressource », réservé en pratique aux outils d'administration.
3. **Créer le realm** si besoin : `GET /admin/realms/tp-ldap` ; si absent, `POST /admin/realms` avec un JSON minimal (realm activé, sans auto-inscription publique).
4. **Fournisseur LDAP** : `GET` des *components* de type stockage utilisateur ; s'il n'existe pas de `providerId` `ldap`, récupération de l'UUID interne du realm puis `POST /admin/realms/.../components` avec le gros JSON de configuration (dont le tableau ci-dessus).
5. **Synchroniser** : `POST .../user-storage/{id}/sync?action=triggerFullSync` pour importer `thomas`, `john`, etc.

Toutes les requêtes d'administration portent l'en-tête `Authorization: Bearer <access_token>`.

7.7 Exécution : copier le script depuis l'annexe

Choix du document : le chapitre explique ; le **code à lancer** n'est pas dupliqué ici. Il figure en entier en **annexe A.10** (identique à `projet/scripts/configure_keycloak_ldap.sh` dans le dépôt Git).

1. Recopier le listing de l'annexe dans un fichier `configure_keycloak_ldap.sh`, puis `chmod +x configure_keycloak_ldap.sh`.
2. Avoir lancé `docker compose up -d --build` et attendre que les services `ldap` et `keycloak` soient prêts.
3. Exécuter sur l'hôte : `bash configure_keycloak_ldap.sh` (depuis n'importe quel répertoire ; adapter `KEYCLOAK_URL` si le port Keycloak diffère).

Si vous clonez le dépôt : `bash projet/scripts/configure_keycloak_ldap.sh`.

7.8 Vérification - requêtes `curl` (aperçu)

Après le script, un utilisateur fédéré doit apparaître dans le realm `tp-ldap`. Exemple de contrôle (même idée que l'API utilisée par le script : token admin, puis liste filtrée) :

```
KEYCLOAK_URL=http://localhost:8090
REALM=tp-ldap
TOKEN=$(curl -s -X POST "${KEYCLOAK_URL}/realms/master/protocol/openid-connect/token"
    \
```

```
  -H "Content-Type: application/x-www-form-urlencoded" \
  --data-urlencode "grant_type=password" \
  --data-urlencode "client_id=admin-cli" \
  --data-urlencode "username=admin" \
  --data-urlencode "password=admin" \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['access_token'])")
curl -s -H "Authorization: Bearer ${TOKEN}" \
  "${KEYCLOAK_URL}/admin/realms/${REALM}/users?username=thomas&exact=true" \
  | python3 -m json.tool
```

Résultat attendu : un tableau JSON contenant **un** utilisateur `thomas`. Faire de même pour `john`. Test manuel : http://localhost:8090/realms/tp-ldap/account - connexion avec login LDAP et mot de passe du DIT (`thomas123` au lab).

Chapitre 8

Réplication : fournisseur / réplica

8.1 Objectif de ce chapitre

Avoir un annuaire **maître** (écritures sur le port `389`) et un **réplica** en lecture seule (port `1389`) qui reçoit les mises à jour en continu. La configuration automatique du TP correspond aux scripts `init_replication_provider.sh` et `init_replication_consumer.sh` (annexes A.7 et A.8).

8.2 Notions

— **syncprov** (fournisseur) : module + overlay qui enregistrent les changements pour les réplicas.

— **syncrepl** (réplica) : le consommateur s'abonne au fournisseur (`refreshAndPersist` = flux continu après la phase initiale).

— `olcServerID` : identifiant numérique unique par nœud (ici `1` et `2`).

— `olcReadOnly` sur la MDB du réplica : les *clients* LDAP ne peuvent pas écrire ; la réplication interne met quand même à jour la copie.

Le compose du TP relie `ldap-replica` au service `ldap` (annexe A.2). Les LDIF ci-dessous sont l'équivalent **manuel**.

8.3 Fournisseur : module `syncprov`, overlay, `olcServerID`, index

Sur `ldap`, après un DIT opérationnel : (1) charger `syncprov.la`, (2) ajouter l'overlay sur `olcDatabase={1}mdb`, (3) fixer `olcServerID: 1`, (4) index `entryUUID`.

```
cat > /tmp/repl_prov_module.ldif <<'EOF'
dn: cn=module\{0\},cn=config
changetype: modify
add: olcModuleLoad
olcModuleLoad: syncprov.la
EOF
docker cp /tmp/repl_prov_module.ldif ldap:/tmp/repl_prov_module.ldif
docker exec ldap ldapmodify -Y EXTERNAL -H ldapi:/// -f /tmp/repl_prov_module.ldif

cat > /tmp/repl_prov_overlay.ldif <<'EOF'
dn: olcOverlay=syncprov,olcDatabase=\{1\}mdb,cn=config
objectClass: olcOverlayConfig
objectClass: olcSyncProvConfig
olcOverlay: syncprov
EOF
docker cp /tmp/repl_prov_overlay.ldif ldap:/tmp/repl_prov_overlay.ldif
```

```
docker exec ldap ldapadd -Y EXTERNAL -H ldapi:/// -f /tmp/repl_prov_overlay.ldif

cat > /tmp/repl_prov_rest.ldif <<'EOF'
dn: cn=config
changetype: modify
add: olcServerID
olcServerID: 1

dn: olcDatabase=\{1\}mdb,cn=config
changetype: modify
add: olcDbIndex
olcDbIndex: entryUUID eq
EOF
docker cp /tmp/repl_prov_rest.ldif ldap:/tmp/repl_prov_rest.ldif
docker exec ldap ldapmodify -Y EXTERNAL -H ldapi:/// -f /tmp/repl_prov_rest.ldif
```

8.4 Réplica : `olcServerID`, lecture seule, `syncrepl`

`olcServerID: 2`, MDB en `ReadOnly` pour les clients, et `olcSyncrepl` vers le fournisseur (`refreshAndPersist`). Sur `ldap-replica` ; adapter URL et mot de passe si besoin.

```
cat > /tmp/repl_consumer.ldif <<'EOF'
dn: cn=config
changetype: modify
add: olcServerID
olcServerID: 2

dn: olcDatabase=\{1\}mdb,cn=config
changetype: modify
add: olcReadOnly
olcReadOnly: TRUE
-
add: olcSyncrepl
olcSyncrepl: rid=001 provider=ldap://ldap:389 bindmethod=simple
    binddn="cn=admin,dc=example,dc=org" credentials=admin
    searchbase="dc=example,dc=org" type=refreshAndPersist retry="60 +" timeout=1
    schemachecking=off scope=sub
EOF
docker cp /tmp/repl_consumer.ldif ldap-replica:/tmp/repl_consumer.ldif
docker exec ldap-replica ldapmodify -Y EXTERNAL -H ldapi:/// -f
    /tmp/repl_consumer.ldif
```

8.5 Fiche de test - propagation et interdiction d'écriture

Objectifs : le réplica voit les données du fournisseur ; une modif sur `389` se retrouve sur `1389` ; une écriture sur `1389` échoue. Le test utilise une `description` temporaire (`MARKER`) puis la supprime sur le fournisseur.

```
ldapsearch -x -H ldap://localhost:1389 -D cn=admin,dc=example,dc=org -w admin \
  -b ou=people,dc=example,dc=org "(uid=thomas)" dn
MARKER="repl-test-$(date +%s)"
ldapmodify -x -H ldap://localhost:389 -D cn=admin,dc=example,dc=org -w admin <<EOF
dn: uid=thomas,ou=people,dc=example,dc=org
changetype: modify
```

```
replace: description
description: $MARKER
EOF
sleep 3
ldapsearch -x -H ldap://localhost:1389 -D cn=admin,dc=example,dc=org -w admin \
  -b uid=thomas,ou=people,dc=example,dc=org -s base description
ldapmodify -x -H ldap://localhost:1389 -D cn=admin,dc=example,dc=org -w admin <<EOF || echo "
dn: uid=john,ou=people,dc=example,dc=org
changetype: modify
replace: description
description: should-fail
EOF
ldapmodify -x -H ldap://localhost:389 -D cn=admin,dc=example,dc=org -w admin <<EOF || true
dn: uid=thomas,ou=people,dc=example,dc=org
changetype: modify
delete: description
EOF
```

Résultat attendu : la première `ldapsearch` sur le port `1389` retourne le DN de `thomas`. Après le `ldapmodify` sur `389`, la recherche sur `1389` affiche `description:` suivi de la valeur `MARKER`. Le `ldapmodify` ciblant `1389` affiche une erreur du type *read-only* ou *Insufficient access*. Le nettoyage sur `389` supprime l'attribut `description`.

Chapitre 9

Méta-annuaire (back-meta)

9.1 Objectif de ce chapitre

Offrir **un seul point LDAP** (port `3389`, suffixe `o=federation`) derrière lequel se cachent **deux annuaires réels** déjà déployés par le compose (`ldap` et `ldap-acme`). C'est la même idée que le script `projet/scripts/init_meta_annuaire.sh` (annexe A.9), déclenché depuis `init_ldap.sh` lorsque `LDAP_SERVICE_ROLE=meta`.

9.2 Vue d'ensemble

Deux backends réels : `dc=example,dc=org` et `dc=acme,dc=com`. Le méta-annuaire présente des **vues** `ou=example-org,o=federation` et `ou=acme-corp,o=federation` ; `suffixmassage` réécrit ces chemins vers les suffixes réels lors des requêtes relayées. `olcDbIDAssertBind` fournit à OpenLDAP un compte pour se lier à chaque annuaire cible.

9.3 Déroulé logique de la configuration

1. Préparer le serveur méta comme pour un nouvel suffixe : retirer la MDB par défaut, données cohérentes sur disque, `slapd` actif.
2. Charger les modules `back_ldap` et `back_meta`.
3. Ajouter une base `olcDatabase=meta` (suffixe `o=federation`, RootDN admin du méta, hash mot de passe).
4. Créer deux entrées `olcMetaSub` : une par annuaire (URI du service Docker, `suffixmassage`, identifiants d'assertion).

Les valeurs concrètes (URIs, DN) correspondent aux variables `LDAP_META_*` du compose (annexe A.2). Schéma LDIF ci-dessous :

```
cat > /tmp/meta_modules.ldif <<'EOF'
dn: cn=module\{0\},cn=config
changetype: modify
add: olcModuleLoad
olcModuleLoad: back_ldap.la
-
add: olcModuleLoad
olcModuleLoad: back_meta.la
EOF

cat > /tmp/meta_db.ldif <<'EOF'
dn: olcDatabase=meta,cn=config
objectClass: olcDatabaseConfig
```

```
objectClass: olcMetaConfig
olcDatabase: meta
olcSuffix: o=federation
olcRootDN: cn=admin,o=federation
olcRootPW: META_ADMIN_HASH
olcDbOnErr: continue
olcDbCancel: abandon
olcDbTFSupport: no
olcAccess: to * by dn.exact=cn=admin,o=federation manage by * read
EOF

# Remplacer META_ADMIN_HASH par : docker exec ldap-meta slappasswd -s admin

cat > /tmp/meta_tgt_example.ldif <<'EOF'
dn: olcMetaSub=\{0\}example,olcDatabase=meta,cn=config
objectClass: olcConfig
objectClass: olcMetaTargetConfig
olcMetaSub: \{0\}example
olcDbURI: ldap://ldap:389/ou=example-org,o=federation
olcDbRewrite: \{0\}suffixmassage "ou=example-org,o=federation" "dc=example,dc=org"
olcDbIDAssertBind: bindmethod=simple binddn="cn=admin,dc=example,dc=org"
    credentials=admin
olcDbChaseReferrals: FALSE
EOF

cat > /tmp/meta_tgt_acme.ldif <<'EOF'
dn: olcMetaSub=\{1\}acme,olcDatabase=meta,cn=config
objectClass: olcConfig
objectClass: olcMetaTargetConfig
olcMetaSub: \{1\}acme
olcDbURI: ldap://ldap-acme:389/ou=acme-corp,o=federation
olcDbRewrite: \{0\}suffixmassage "ou=acme-corp,o=federation" "dc=acme,dc=com"
olcDbIDAssertBind: bindmethod=simple binddn="cn=admin,dc=acme,dc=com"
    credentials=admin
olcDbChaseReferrals: FALSE
EOF
# Puis : docker cp ... ldap-meta:/tmp/ ... et ldapmodify / ldapadd -Y EXTERNAL -H
    ldapi:///
# dans l'ordre modules -> base meta -> cibles (slapd actif).
```

Remarque : Ordre et redémarrages de `slapd` sont sensibles ; ce bloc est une **référence LDIF** à appliquer avec `ldapmodify`/`ldapadd`.

9.4 Contrôle des contextes

```
ldapsearch -x -H ldap://localhost:3389 -D cn=admin,o=federation -w admin \
  -b "" -s base namingContexts
```

Résultat attendu : la réponse contient au minimum le suffixe virtuel `o=federation` (et éventuellement d'autres contextes selon la configuration affichée par le serveur).

9.5 Fiche de test - deux sources pour `uid=thomas`

Les deux annuaires contiennent un `uid=thomas` avec une adresse `mail` différente.

```
ldapsearch -x -H ldap://localhost:3389 -D cn=admin,o=federation -w admin \
  -b ou=people,ou=example-org,o=federation -s sub "(uid=thomas)" mail
ldapsearch -x -H ldap://localhost:3389 -D cn=admin,o=federation -w admin \
  -b ou=people,ou=acme-corp,o=federation -s sub "(uid=thomas)" mail
ldapsearch -x -H ldap://localhost:3389 -D cn=admin,o=federation -w admin \
  -b o=federation -s sub "(uid=thomas)" dn
# Compter les lignes dn: uid=thomas (attendu : 2)
```

Résultat attendu : `thomas@example.org` sous `example-org`, `thomas@acme.com` sous `acme-corp`.

Annexe A

Sources complètes du laboratoire

Fichiers pour monter la stack du TP : compose, image Docker, point d'entrée du conteneur, puis les petits fichiers shell copiés dans l'image (build). Le guide principal décrit surtout les **commandes LDAP** à la main ; cette annexe sert à disposer d'une copie des fichiers du projet. Compiler avec `annexe-code-inclus.tex` dans le même dossier que ce `.tex`.

A.1 Vue d'ensemble des fichiers

Fichier	Rôle
`docker-compose.yml`	Services, ports, volumes, variables.
`Dockerfile` / `entrypoint.sh`	Image avec `slapd`, démarrage du conteneur.
Autres fichiers `.sh` listés plus bas	Nécessaires au `docker build` tel que défini dans le `Dockerfile`.

A.2 `docker-compose.yml`

```
name: tp-ldap

services:
  ldap:
    build:
      context: .
      dockerfile: docker/Dockerfile
    container_name: ldap
    environment:
      LDAP_ORGANISATION: Example Org
      LDAP_DOMAIN: example.org
      LDAP_BASE_DN: dc=example,dc=org
      LDAP_ADMIN_PASSWORD: admin
      LDAP_TLS: "false"
    ports:
      - "389:389"
    volumes:
      - ldap_data:/var/lib/ldap
      - ldap_config:/etc/ldap/slapd.d
    restart: "no"
    stop_grace_period: 15s
    healthcheck:
      # Fichier écrit après tous les scripts init.d ; puis DIT + syncprov (réplication).
      test:
        [
          "CMD-SHELL",
          "test -f /container/.ldap-init-complete && ldapsearch -x -H ldap://127.0.0.1:389 -D cn=admin,dc=exampl
        ]
```

```
    interval: 5s
    timeout: 5s
    retries: 12
    start_period: 30s

ldap-replica:
  build:
    context: .
    dockerfile: docker/Dockerfile
  container_name: ldap-replica
  environment:
    LDAP_ORGANISATION: Example Org
    LDAP_DOMAIN: example.org
    LDAP_BASE_DN: dc=example,dc=org
    LDAP_ADMIN_PASSWORD: admin
    LDAP_TLS: "false"
    LDAP_SERVICE_ROLE: consumer
    LDAP_REPLICATION_PROVIDER_URI: ldap://ldap:389
  ports:
    - "1389:389"
  volumes:
    - ldap_replica_data:/var/lib/ldap
    - ldap_replica_config:/etc/ldap/slapd.d
  depends_on:
    ldap:
      condition: service_healthy
  healthcheck:
    test:
      [
        "CMD-SHELL",
        "test -f /container/.ldap-init-complete && ldapsearch -x -H ldap://127.0.0.1:389 -b '' -s base namingC
      ]
    interval: 5s
    timeout: 5s
    retries: 15
    start_period: 60s
  restart: "no"
  stop_grace_period: 15s

# Second annuaire (entité distincte) pour le méta-annuaire - même image, autre suffixe.
ldap-acme:
  build:
    context: .
    dockerfile: docker/Dockerfile
  container_name: ldap-acme
  environment:
    LDAP_ORGANISATION: Acme Corp
    LDAP_DOMAIN: acme.com
    LDAP_BASE_DN: dc=acme,dc=com
    LDAP_ADMIN_PASSWORD: admin
    LDAP_TLS: "false"
  ports:
    - "2389:389"
  volumes:
    - ldap_acme_data:/var/lib/ldap
    - ldap_acme_config:/etc/ldap/slapd.d
  depends_on:
    ldap:
      condition: service_healthy
  healthcheck:
    test:
      [
        "CMD-SHELL",
```

```
        "test -f /container/.ldap-init-complete && ldapsearch -x -H ldap://127.0.0.1:389 -D cn=admin,dc=acme,d
      ]
    interval: 5s
    timeout: 5s
    retries: 12
    start_period: 30s
  restart: "no"
  stop_grace_period: 15s

# Méta-annuaire (back-meta) : agrège ldap (example.org) et ldap-acme (acme.com).
ldap-meta:
  build:
    context: .
    dockerfile: docker/Dockerfile
  container_name: ldap-meta
  environment:
    LDAP_SERVICE_ROLE: meta
    LDAP_ADMIN_PASSWORD: admin
    LDAP_TLS: "false"
    LDAP_META_SUFFIX: o=federation
    LDAP_META_EXAMPLE_URI: ldap://ldap:389
    LDAP_META_EXAMPLE_SUFFIX: dc=example,dc=org
    LDAP_META_EXAMPLE_VIRTUAL: ou=example-org,o=federation
    LDAP_META_ACME_URI: ldap://ldap-acme:389
    LDAP_META_ACME_SUFFIX: dc=acme,dc=com
    LDAP_META_ACME_VIRTUAL: ou=acme-corp,o=federation
  ports:
    - "3389:389"
  volumes:
    - ldap_meta_data:/var/lib/ldap
    - ldap_meta_config:/etc/ldap/slapd.d
  depends_on:
    ldap:
      condition: service_healthy
    ldap-acme:
      condition: service_healthy
  healthcheck:
    test:
      [
        "CMD-SHELL",
        "test -f /container/.ldap-init-complete && ldapsearch -x -H ldap://127.0.0.1:389 -D cn=admin,o=federat
      ]
    interval: 5s
    timeout: 5s
    retries: 15
    start_period: 45s
  restart: "no"
  stop_grace_period: 15s

keycloak:
  image: quay.io/keycloak/keycloak:26.2
  container_name: keycloak
  command: ["start-dev", "--http-port=8080", "--hostname-strict=false"]
  environment:
    KC_BOOTSTRAP_ADMIN_USERNAME: admin
    KC_BOOTSTRAP_ADMIN_PASSWORD: admin
    KC_HEALTH_ENABLED: "true"
    KC_HTTP_MANAGEMENT_PORT: "9000"
  ports:
    # 8090 / 9001 : éviter le conflit avec un service déjà sur 8080 (ex. autre stack).
    - "8090:8080"
    - "9001:9000"
  depends_on:
```

```
      ldap:
        condition: service_healthy
    restart: "no"

volumes:
  ldap_data: {}
  ldap_config: {}
  ldap_replica_data: {}
  ldap_replica_config: {}
  ldap_acme_data: {}
  ldap_acme_config: {}
  ldap_meta_data: {}
  ldap_meta_config: {}
```

A.3 docker/Dockerfile

```
FROM debian:12-slim

ENV DEBIAN_FRONTEND=noninteractive \
    LDAP_ORGANISATION="Example Org" \
    LDAP_DOMAIN="example.org" \
    LDAP_BASE_DN="dc=example,dc=org" \
    LDAP_ADMIN_PASSWORD="admin" \
    LDAP_TLS="false"

# slapd + clients ; NSS/PAM + nslcd/nscd pour l'objectif 4 (getent).
RUN apt-get update \
    && apt-get install -y --no-install-recommends \
       slapd ldap-utils tini bash coreutils procps ca-certificates sysvinit-utils \
       libpam-ldap libnss-ldap nslcd nscd \
    && rm -rf /var/lib/apt/lists/*

RUN mkdir -p /container/init.d /var/lib/ldap /etc/ldap/slapd.d \
    && chown -R openldap:openldap /var/lib/ldap /etc/ldap/slapd.d

COPY --chown=openldap:openldap scripts/init_ldap.sh /container/init.d/init_ldap.sh
COPY --chown=openldap:openldap scripts/init_ldap_linux_integration.sh /container/init.d/init_ldap_linux_integrat
COPY --chown=openldap:openldap scripts/init_replication_provider.sh /container/init.d/init_replication_provider.
COPY --chown=openldap:openldap scripts/init_replication_consumer.sh /container/init.d/init_replication_consumer.
COPY --chown=root:root scripts/init_meta_annuaire.sh /container/init_meta_annuaire.sh
COPY docker/entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh /container/init_meta_annuaire.sh /container/init.d/init_ldap.sh /container/init.d/in
    /container/init.d/init_replication_provider.sh /container/init.d/init_replication_consumer.sh

EXPOSE 389

VOLUME ["/var/lib/ldap", "/etc/ldap/slapd.d"]

ENTRYPOINT ["/usr/bin/tini", "--", "/entrypoint.sh"]
```

A.4 docker/entrypoint.sh

```
#!/usr/bin/env bash
set -euo pipefail

# Script d'entrée pour le conteneur LDAP
# Ce script démarre slapd et exécute les scripts d'initialisation

echo "[entrypoint] Démarrage du conteneur LDAP..."

# Les volumes Docker montés sur /var/lib/ldap et /etc/ldap/slapd.d sont vides
```

```
# et souvent propriété de root : slapd (-u openldap) ne peut pas y accéder.
chown -R openldap:openldap /var/lib/ldap /etc/ldap/slapd.d

# Initialisation de la configuration slapd si nécessaire
# Cette étape crée la structure de base cn=config
if [ ! -d "/etc/ldap/slapd.d" ] || [ -z "$(ls -A /etc/ldap/slapd.d || true)" ]; then
  echo "[entrypoint] Initialisation de la configuration slapd..."
  slaptest -F /etc/ldap/slapd.d -f /dev/null >/dev/null 2>&1 || true
  chown -R openldap:openldap /etc/ldap/slapd.d
fi

# Vérifier si slapd est déjà en cours d'exécution
if pgrep slapd >/dev/null 2>&1; then
    echo "[entrypoint] slapd déjà en cours d'exécution"
else
    # Démarrage du serveur slapd en arrière-plan.
    # slapd se daemonise (fork) : le PID du shell ($!) meurt tout de suite, on vérifie la vie du
    # service avec pgrep slapd, pas avec kill -0 $!.
    echo "[entrypoint] Lancement de slapd..."
    slapd -h "ldap:/// ldapi:///" -u openldap -g openldap -F /etc/ldap/slapd.d &
fi

# Attendre que slapd écoute (processus slapd + socket ldapi + LDAP TCP).
echo "[entrypoint] Attente du socket ldapi et du port LDAP 389..."
ready=0
for i in {1..60}; do
  if ! pgrep slapd >/dev/null 2>&1; then
    echo "[entrypoint] ERREUR: aucun processus slapd. Vérifiez les droits sur /etc/ldap/slapd.d et /var/lib/ldap
    exit 1
  fi
  if { [ -S /var/run/slapd/ldapi ] || [ -S /run/slapd/ldapi ]; } && bash -c ":>/dev/tcp/127.0.0.1/389" 2>/dev/nu
    echo "[entrypoint] slapd prêt (ldapi + port 389)"
    ready=1
    break
  fi
  sleep 1
done
if [ "$ready" != 1 ]; then
  echo "[entrypoint] ERREUR: slapd ne semble pas joignable après 60 s."
  exit 1
fi

# Exécution des scripts d'initialisation (ordre fixe : pas de préfixe numérique sur les noms)
if [ -d "/container/init.d" ]; then
  echo "[entrypoint] Exécution des scripts d'initialisation..."
  for name in init_ldap.sh init_ldap_linux_integration.sh init_replication_provider.sh init_replication_consumer
    script="/container/init.d/$name"
    [ -e "$script" ] || continue
    echo "[entrypoint] Exécution: $script"
    if bash "$script"; then
      echo "[entrypoint] Terminé OK: $script"
    else
      rc=$?
      echo "[entrypoint] AVERTISSEMENT: $script a retourné le code $rc (poursuite du démarrage)"
    fi
  done
  echo "[entrypoint] Scripts d'initialisation terminés"
  touch /container/.ldap-init-complete
fi

# Arrêt propre : docker stop envoie SIGTERM à PID 1
# Éviter « sleep infinity » en boucle : l'arrêt peut rester bloqué très longtemps en « Stopping »
_shutdown() {
```

```
  echo "[entrypoint] Arrêt demandé (SIGTERM/SIGINT)..."
  pkill -u openldap -TERM slapd 2>/dev/null || pkill -TERM slapd 2>/dev/null || true
  local i
  for i in $(seq 1 40); do
    pgrep slapd >/dev/null 2>&1 || break
    sleep 0.25
  done
  pkill -u openldap -KILL slapd 2>/dev/null || pkill -KILL slapd 2>/dev/null || true
  exit 0
}
trap _shutdown TERM INT

echo "[entrypoint] Conteneur prêt (docker stop pour arrêter)..."
tail -f /dev/null &
wait $!
```

A.5 Script scripts/init_ldap.sh

```
#!/usr/bin/env bash
# Script d'initialisation LDAP
# Ce script configure la base mdb, crée le DIT et configure les ACL

# Désactiver l'arrêt sur erreur pour permettre la continuation
set -uo pipefail

# Variables d'environnement attendues:
# - LDAP_BASE_DN
# - LDAP_ORGANISATION
# - LDAP_DOMAIN
# - LDAP_ADMIN_PASSWORD (utilisée pour le rootDN de la base)
# - LDAP_SERVICE_ROLE : provider (défaut) = DIT complet ; consumer = base mdb + ACL, données via syncrepl

BASE_DN=${LDAP_BASE_DN:-"dc=polytech,dc=fr"}
ROLE=${LDAP_SERVICE_ROLE:-provider}
ORG=${LDAP_ORGANISATION:-"Polytech"}
DOMAIN=${LDAP_DOMAIN:-"polytech.fr"}
ADMIN_PASS=${LDAP_ADMIN_PASSWORD:-"admin123"}

export LDAPTLS_REQCERT=never # pour ne pas avoir à fournir de certificat lors de la connexion

# Attendre que slapd réponde (ldapi)
for i in {1..30}; do
  if ldapwhoami -H ldapi:/// -Y EXTERNAL >/dev/null 2>&1; then
    break
  fi
  sleep 1
done

if [ "$ROLE" = "meta" ]; then
  echo "[init_ldap] Rôle meta : méta-annuaire (pas de DIT mdb local)."
  bash /container/init_meta_annuaire.sh
  exit $?
fi

# Supprimer la base mdb par défaut et créer une nouvelle
echo "[init_ldap] Suppression de la base mdb par défaut..."
ldapmodify -H ldapi:/// -Y EXTERNAL <<EOF
dn: olcDatabase={1}mdb,cn=config
changetype: delete
EOF
echo "[init_ldap] Base mdb par défaut supprimée"
```

```
# Les fichiers sous /var/lib/ldap gardent sinon l'ancien suffixe (ex. dc=nodomain) alors que
# cn=config pointe déjà vers LDAP_BASE_DN → syncrepl / slapcat incohérents.
echo "[init_ldap] Purge des fichiers mdb et redémarrage de slapd..."
pkill -u openldap -TERM slapd 2>/dev/null || pkill -TERM slapd 2>/dev/null || true
for _ in $(seq 1 40); do
  pgrep slapd >/dev/null 2>&1 || break
  sleep 0.25
done
pkill -u openldap -KILL slapd 2>/dev/null || pkill -KILL slapd 2>/dev/null || true
find /var/lib/ldap -mindepth 1 -delete 2>/dev/null || true
chown openldap:openldap /var/lib/ldap
slapd -h "ldap:/// ldapi:///" -u openldap -g openldap -F /etc/ldap/slapd.d &
for _ in $(seq 1 60); do
  if ldapwhoami -H ldapi:/// -Y EXTERNAL >/dev/null 2>&1; then
    break
  fi
  sleep 1
done

# Créer une nouvelle base mdb avec la configuration correcte
echo "[init_ldap] Création de la base mdb pour suffix=$BASE_DN..."
HASH=$(slappasswd -s "$ADMIN_PASS")
cat > /tmp/new-db.ldif <<EOF
dn: olcDatabase=mdb,cn=config
objectClass: olcDatabaseConfig
objectClass: olcMdbConfig
olcDatabase: mdb
olcSuffix: $BASE_DN
olcRootDN: cn=admin,$BASE_DN
olcRootPW: $HASH
olcDbDirectory: /var/lib/ldap
olcDbIndex: objectClass eq
olcDbIndex: cn,sn,uid eq
olcDbMaxSize: 1073741824
EOF
ldapadd -H ldapi:/// -Y EXTERNAL -f /tmp/new-db.ldif
echo "[init_ldap] Base mdb créée avec suffix=$BASE_DN"

if [ "$ROLE" = "consumer" ]; then
  echo "[init_ldap] Rôle consumer : pas de création locale du DIT (réplication depuis le fournisseur)."
else
# DIT: base + ou=people + ou=groups + groupe admin_ldap + groupe developers + utilisateurs thomas et john
cat > /tmp/dit.ldif <<EOF
version: 1

dn: $BASE_DN
objectClass: top
objectClass: dcObject
objectClass: organization
o: $ORG
dc: ${DOMAIN%%.*}

dn: ou=people,$BASE_DN
objectClass: top
objectClass: organizationalUnit
ou: people

dn: ou=groups,$BASE_DN
objectClass: top
objectClass: organizationalUnit
ou: groups

dn: cn=admin_ldap,ou=groups,$BASE_DN
```

```
objectClass: top
objectClass: groupOfNames
cn: admin_ldap
member: cn=dummy,$BASE_DN

dn: cn=developers,ou=groups,$BASE_DN
objectClass: top
objectClass: groupOfNames
cn: developers
member: cn=dummy,$BASE_DN

dn: cn=admin_keycloak,ou=groups,$BASE_DN
objectClass: top
objectClass: groupOfNames
cn: admin_keycloak
member: cn=dummy,$BASE_DN

dn: uid=thomas,ou=people,$BASE_DN
objectClass: top
objectClass: inetOrgPerson
cn: Thomas
sn: Thomas
uid: thomas
mail: thomas@$DOMAIN
userPassword: $(slappasswd -s thomas123)

dn: uid=john,ou=people,$BASE_DN
objectClass: top
objectClass: inetOrgPerson
cn: John
sn: John
uid: john
mail: john@$DOMAIN
userPassword: $(slappasswd -s john123)
EOF

# Ajouter la structure si absente (bind simple sur rootDN)
echo "[init_ldap] Vérification de l'existence du DIT..."
if ! ldapsearch -x -H ldap:/// -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" -b "$BASE_DN" -s base dn >/dev/null 2>&1;
  echo "[init_ldap] Création DIT de base..."
  ldapadd -x -H ldap:/// -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" -f /tmp/dit.ldif || echo "[init_ldap] Erreur lo
  echo "[init_ldap] DIT créé"
else
  echo "[init_ldap] DIT déjà présent"
fi
echo "[init_ldap] Étape DIT terminée"

# Ajouter thomas au groupe admin_ldap et john au groupe developers
echo "[init_ldap] Ajout de thomas au groupe admin_ldap..."
cat > /tmp/add-thomas-to-admin.ldif <<EOF
dn: cn=admin_ldap,ou=groups,$BASE_DN
changetype: modify
delete: member
member: cn=dummy,$BASE_DN
-
add: member
member: uid=thomas,ou=people,$BASE_DN
EOF
ldapmodify -x -H ldap:/// -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" -f /tmp/add-thomas-to-admin.ldif || echo "[ini
echo "[init_ldap] Thomas ajouté au groupe admin_ldap"

echo "[init_ldap] Ajout de john au groupe developers..."
cat > /tmp/add-john-to-developers.ldif <<EOF
```

```
dn: cn=developers,ou=groups,$BASE_DN
changetype: modify
delete: member
member: cn=dummy,$BASE_DN
-
add: member
member: uid=john,ou=people,$BASE_DN
EOF
ldapmodify -x -H ldap:/// -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" -f /tmp/add-john-to-developers.ldif || echo "[
echo "[init_ldap] John ajouté au groupe developers"

echo "[init_ldap] Ajout de thomas au groupe admin_keycloak..."
cat > /tmp/add-thomas-to-admin-keycloak.ldif <<EOF
dn: cn=admin_keycloak,ou=groups,$BASE_DN
changetype: modify
delete: member
member: cn=dummy,$BASE_DN
-
add: member
member: uid=thomas,ou=people,$BASE_DN
EOF
ldapmodify -x -H ldap:/// -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" -f /tmp/add-thomas-to-admin-keycloak.ldif || e
echo "[init_ldap] Thomas ajouté au groupe admin_keycloak"

# Tentative de suppression d'un éventuel entry cn=admin (peu probable qu'il existe)
if ldapsearch -x -H ldap:/// -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" -b "$BASE_DN" "(cn=admin)" dn | grep -q "^d
  echo "[init_ldap] Suppression cn=admin pour discrétisation..."
  cat > /tmp/delete-admin.ldif <<EOF
version: 1

dn: cn=admin,$BASE_DN
changetype: delete
EOF
  ldapmodify -x -H ldap:/// -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" -f /tmp/delete-admin.ldif || echo "[init_lda
fi

fi # fin rôle provider (DIT)

# Configuration des ACL de base pour permettre l'accès
echo "[init_ldap] Configuration des ACL de base..."
cat > /tmp/acl-basic.ldif <<EOF
dn: olcDatabase={1}mdb,cn=config
changetype: modify
add: olcAccess
olcAccess: to * by dn.exact=cn=admin,$BASE_DN manage by * read
EOF
ldapmodify -H ldapi:/// -Y EXTERNAL -f /tmp/acl-basic.ldif || echo "[init_ldap] Erreur lors de la configuration

# Configuration des ACL avancées pour le groupe admin_ldap
echo "[init_ldap] Configuration des ACL avancées pour admin_ldap..."
cat > /tmp/acl-advanced.ldif <<EOF
dn: olcDatabase={1}mdb,cn=config
changetype: modify
add: olcAccess
olcAccess: to attrs=userPassword by group.exact=cn=admin_ldap,ou=groups,$BASE_DN write by self write by anonymou
olcAccess: to dn.base="" by * read
olcAccess: to * by group.exact=cn=admin_ldap,ou=groups,$BASE_DN manage by self write by users read by * none
EOF
ldapmodify -H ldapi:/// -Y EXTERNAL -f /tmp/acl-advanced.ldif || echo "[init_ldap] Erreur lors de la configurati

echo "[init_ldap] Structure DIT et ACL configurées."
```

A.6 Script scripts/init_ldap_linux_integration.sh

```
#!/usr/bin/env bash
# Script d'initialisation de l'intégration LDAP-Linux
# Ce script configure PAM, SSSD et NSS pour l'authentification LDAP

# Variables d'environnement
BASE_DN=${LDAP_BASE_DN:-"dc=example,dc=org"}
DOMAIN=${LDAP_DOMAIN:-"example.org"}
ADMIN_PASS=${LDAP_ADMIN_PASSWORD:-"admin"}

if [ "${LDAP_SERVICE_ROLE:-provider}" = "consumer" ]; then
  echo "[init_ldap_linux] Ignoré sur réplica (LDAP_SERVICE_ROLE=consumer)."
  exit 0
fi

if [ "${LDAP_SERVICE_ROLE:-provider}" = "meta" ]; then
  echo "[init_ldap_linux] Ignoré sur méta-annuaire (LDAP_SERVICE_ROLE=meta)."
  exit 0
fi

echo "[init_ldap_linux] Début de la configuration de l'intégration LDAP-Linux..."

# Attendre que slapd soit prêt
for i in {1..30}; do
  if ldapwhoami -H ldapi:/// -Y EXTERNAL >/dev/null 2>&1; then
    break
  fi
  sleep 1
done

# Configuration de NSS (Name Service Switch)
echo "[init_ldap_linux] Configuration de NSS..."
cat > /etc/nsswitch.conf <<EOF
# /etc/nsswitch.conf
# Configuration pour l'intégration LDAP

passwd:         files ldap
group:          files ldap
shadow:         files ldap
gshadow:        files

hosts:          files dns
networks:       files

protocols:      db files
services:       db files
ethers:         db files
rpc:            db files

netgroup:       ldap
EOF

# Configuration de libnss-ldap
echo "[init_ldap_linux] Configuration de libnss-ldap..."
cat > /etc/ldap/ldap.conf <<EOF
# Configuration LDAP pour NSS
BASE $BASE_DN
URI ldap://localhost:389
TLS_REQCERT never
EOF

# Configuration de PAM pour LDAP
```

```
echo "[init_ldap_linux] Configuration de PAM..."
cat > /etc/pam.d/common-auth <<EOF
# Configuration PAM pour l'authentification LDAP
auth    sufficient      pam_ldap.so
auth    sufficient      pam_unix.so nullok_secure use_first_pass
auth    required        pam_deny.so
EOF

cat > /etc/pam.d/common-account <<EOF
# Configuration PAM pour la gestion des comptes
account sufficient      pam_ldap.so
account sufficient      pam_unix.so
account required        pam_deny.so
EOF

cat > /etc/pam.d/common-password <<EOF
# Configuration PAM pour les mots de passe
password        sufficient      pam_ldap.so
password        sufficient      pam_unix.so nullok obscure min=4 max=8 md5
password        required        pam_deny.so
EOF

cat > /etc/pam.d/common-session <<EOF
# Configuration PAM pour les sessions
session required        pam_mkhomedir.so skel=/etc/skel umask=0022
session sufficient      pam_ldap.so
session sufficient      pam_unix.so
session required        pam_deny.so
EOF

# Configuration de nslcd (plus simple et fiable que SSSD dans un conteneur)
echo "[init_ldap_linux] Configuration de nslcd..."
cat > /etc/nslcd.conf <<EOF
# Configuration nslcd pour LDAP
uri ldap://localhost:389
base $BASE_DN
ldap_version 3
binddn cn=admin,$BASE_DN
bindpw $ADMIN_PASS
filter passwd (objectClass=posixAccount)
filter group (objectClass=posixGroup)
filter shadow (objectClass=shadowAccount)
map passwd homeDirectory homeDirectory
map passwd uidNumber uidNumber
map passwd gidNumber gidNumber
map passwd loginShell loginShell
map passwd gecos gecos
map group gidNumber gidNumber
map group memberUid memberUid
EOF

# Permissions pour nslcd
chmod 600 /etc/nslcd.conf
chown root:root /etc/nslcd.conf

# Configuration de libpam-ldap
echo "[init_ldap_linux] Configuration de libpam-ldap..."
cat > /etc/ldap.conf <<EOF
# Configuration pour libpam-ldap
base $BASE_DN
uri ldap://localhost:389
ldap_version 3
rootbinddn cn=admin,$BASE_DN
```

```
binddn cn=admin,$BASE_DN
bindpw $ADMIN_PASS
pam_password crypt
nss_base_passwd ou=people,$BASE_DN
nss_base_shadow ou=people,$BASE_DN
nss_base_group ou=groups,$BASE_DN
nss_map_objectclass posixAccount user
nss_map_objectclass shadowAccount user
nss_map_objectclass posixGroup group
nss_map_attribute uid sAMAccountName
nss_map_attribute homeDirectory unixHomeDirectory
nss_map_attribute shadowLastChange pwdLastSet
nss_map_objectclass posixAccount user
nss_map_objectclass shadowAccount user
nss_map_objectclass posixGroup group
nss_map_attribute uid sAMAccountName
nss_map_attribute homeDirectory unixHomeDirectory
nss_map_attribute shadowLastChange pwdLastSet
pam_login_attribute uid
pam_member_attribute member
pam_filter objectclass=inetOrgPerson
pam_password exop
EOF

# Ajout des attributs POSIX aux utilisateurs LDAP
echo "[init_ldap_linux] Ajout des attributs POSIX aux utilisateurs..."
cat > /tmp/add_posix_attrs.ldif <<EOF
dn: uid=thomas,ou=people,$BASE_DN
changetype: modify
add: objectClass
objectClass: posixAccount
objectClass: shadowAccount
-
add: uidNumber
uidNumber: 1001
-
add: gidNumber
gidNumber: 1001
-
add: homeDirectory
homeDirectory: /home/thomas
-
add: loginShell
loginShell: /bin/bash
-
add: gecos
gecos: Thomas Thomas

dn: uid=john,ou=people,$BASE_DN
changetype: modify
add: objectClass
objectClass: posixAccount
objectClass: shadowAccount
-
add: uidNumber
uidNumber: 1002
-
add: gidNumber
gidNumber: 1002
-
add: homeDirectory
homeDirectory: /home/john
-
```

```
add: loginShell
loginShell: /bin/bash
-
add: gecos
gecos: John John
EOF

ldapmodify -x -H ldap:/// -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" -f /tmp/add_posix_attrs.ldif || echo "[init_ld

# Conversion des groupes de groupOfNames à posixGroup
echo "[init_ldap_linux] Conversion des groupes en posixGroup..."
# Vérifier d'abord si les groupes existent et ont des membres
# Supprimer d'abord les membres existants (s'ils existent)
cat > /tmp/convert_groups_to_posix.ldif <<EOF
dn: cn=admin_ldap,ou=groups,$BASE_DN
changetype: modify
delete: objectClass
objectClass: groupOfNames
-
delete: member
EOF

# Ajouter les membres s'ils existent
if ldapsearch -x -H ldap:/// -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" -b "cn=admin_ldap,ou=groups,$BASE_DN" "(mem
    # Supprimer tous les membres existants
    ldapsearch -x -H ldap:/// -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" -b "cn=admin_ldap,ou=groups,$BASE_DN" "(me
fi

cat >> /tmp/convert_groups_to_posix.ldif <<EOF
-
add: objectClass
objectClass: posixGroup
-
add: gidNumber
gidNumber: 1001
-
add: memberUid
memberUid: thomas

dn: cn=developers,ou=groups,$BASE_DN
changetype: modify
delete: objectClass
objectClass: groupOfNames
-
delete: member
EOF

if ldapsearch -x -H ldap:/// -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" -b "cn=developers,ou=groups,$BASE_DN" "(mem
    ldapsearch -x -H ldap:/// -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" -b "cn=developers,ou=groups,$BASE_DN" "(me
fi

cat >> /tmp/convert_groups_to_posix.ldif <<EOF
-
add: objectClass
objectClass: posixGroup
-
add: gidNumber
gidNumber: 1002
-
add: memberUid
memberUid: john
EOF
```

```
ldapmodify -x -H ldap:/// -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" -f /tmp/convert_groups_to_posix.ldif 2>&1 || {
    echo "[init_ldap_linux] Tentative de conversion des groupes..."
    # Si la conversion échoue, essayer une approche plus simple : supprimer et recréer
    echo "[init_ldap_linux] Suppression et recréation des groupes en posixGroup..."
    # Supprimer les groupes existants
    ldapdelete -x -H ldap:/// -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" "cn=admin_ldap,ou=groups,$BASE_DN" 2>/dev/
    ldapdelete -x -H ldap:/// -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" "cn=developers,ou=groups,$BASE_DN" 2>/dev/
    sleep 1
    # Recréer les groupes en posixGroup
    cat > /tmp/recreate_posix_groups.ldif <<EOF
dn: cn=admin_ldap,ou=groups,$BASE_DN
objectClass: top
objectClass: posixGroup
cn: admin_ldap
gidNumber: 1001
memberUid: thomas

dn: cn=developers,ou=groups,$BASE_DN
objectClass: top
objectClass: posixGroup
cn: developers
gidNumber: 1002
memberUid: john
EOF
    ldapadd -x -H ldap:/// -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" -f /tmp/recreate_posix_groups.ldif 2>&1 && ec
}

# Démarrage des services
echo "[init_ldap_linux] Démarrage des services..."
# Démarrer nslcd
service nslcd restart || service nslcd start
# Attendre que nslcd soit prêt
sleep 2
# Redémarrer nscd pour recharger la configuration et vider le cache
service nscd restart || service nscd start
# Vider le cache nscd pour forcer la relecture depuis LDAP
nscd -i passwd 2>/dev/null || true
nscd -i group 2>/dev/null || true
sleep 1

# Test de l'intégration
echo "[init_ldap_linux] Test de l'intégration..."
echo "Test de résolution des utilisateurs:"
getent passwd thomas
getent passwd john

echo "Test de résolution des groupes:"
getent group admin_ldap
getent group developers

echo "[init_ldap_linux] Configuration de l'intégration LDAP-Linux terminée."
```

A.7 Script scripts/init_replication_provider.sh

```
#!/usr/bin/env bash
# Fournisseur de réplication : overlay syncprov + olcServerID (OpenLDAP 2.6).
set -uo pipefail

if [ "${LDAP_SERVICE_ROLE:-provider}" = "consumer" ]; then
  exit 0
fi
```

```
if [ "${LDAP_SERVICE_ROLE:-provider}" = "meta" ]; then
  exit 0
fi

for _ in $(seq 1 30); do
  if ldapwhoami -H ldapi:/// -Y EXTERNAL >/dev/null 2>&1; then
    break
  fi
  sleep 1
done

if ldapsearch -H ldapi:/// -Y EXTERNAL -b cn=config -s sub -LLL '(objectClass=olcSyncProvConfig)' dn 2>/dev/null
  echo "[init_replication_provider] Overlay syncprov déjà présent."
else
  echo "[init_replication_provider] Chargement module syncprov.la sur cn=module{0}..."
  ldapmodify -H ldapi:/// -Y EXTERNAL <<'EOF' 2>/dev/null || true
dn: cn=module{0},cn=config
changetype: modify
add: olcModuleLoad
olcModuleLoad: syncprov.la
EOF
  echo "[init_replication_provider] Ajout overlay syncprov sur mdb {1}..."
  ldapadd -H ldapi:/// -Y EXTERNAL <<'EOF'
dn: olcOverlay=syncprov,olcDatabase={1}mdb,cn=config
objectClass: olcOverlayConfig
objectClass: olcSyncProvConfig
olcOverlay: syncprov
EOF
fi

if ldapsearch -H ldapi:/// -Y EXTERNAL -b cn=config -s base -LLL olcServerID 2>/dev/null | grep -q "^olcServerID
  echo "[init_replication_provider] olcServerID=1 déjà présent."
else
  echo "[init_replication_provider] Ajout olcServerID=1..."
  ldapmodify -H ldapi:/// -Y EXTERNAL <<'EOF' 2>/dev/null || true
dn: cn=config
changetype: modify
add: olcServerID
olcServerID: 1
EOF
fi

echo "[init_replication_provider] Index entryUUID (recommandé pour syncrepl)..."
ldapmodify -H ldapi:/// -Y EXTERNAL <<'EOF' 2>/dev/null || true
dn: olcDatabase={1}mdb,cn=config
changetype: modify
add: olcDbIndex
olcDbIndex: entryUUID eq
EOF

echo "[init_replication_provider] Terminé."
```

A.8 Script scripts/init_replication_consumer.sh

```
#!/usr/bin/env bash
# Réplica en lecture seule : olcSyncrepl + olcReadOnly (les écritures client sont refusées, pas la synchro).
set -uo pipefail

if [ "${LDAP_SERVICE_ROLE:-provider}" != "consumer" ]; then
  exit 0
fi
```

```
BASE_DN=${LDAP_BASE_DN:-dc=example,dc=org}
ADMIN_PASS=${LDAP_ADMIN_PASSWORD:-admin}
PROVIDER_URI=${LDAP_REPLICATION_PROVIDER_URI:-ldap://ldap:389}

for _ in $(seq 1 30); do
  if ldapwhoami -H ldapi:/// -Y EXTERNAL >/dev/null 2>&1; then
    break
  fi
  sleep 1
done

echo "[init_replication_consumer] Attente du fournisseur ${PROVIDER_URI}..."
ready=0
for _ in $(seq 1 90); do
  if ldapsearch -x -H "$PROVIDER_URI" -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" -b "$BASE_DN" -s base dn >/dev/nul
    && ldapsearch -x -H "$PROVIDER_URI" -D "cn=admin,$BASE_DN" -w "$ADMIN_PASS" -b "ou=people,$BASE_DN" -s base
    ready=1
    break
  fi
  sleep 2
done
if [ "$ready" != 1 ]; then
  echo "[init_replication_consumer] AVERTISSEMENT : fournisseur peu joignable ; syncrepl pourrait échouer au dém
fi

if ! ldapsearch -H ldapi:/// -Y EXTERNAL -b cn=config -s base -LLL olcServerID 2>/dev/null | grep -q "^olcServer
  echo "[init_replication_consumer] olcServerID=2 sur cn=config..."
  ldapmodify -H ldapi:/// -Y EXTERNAL <<'EOF' 2>/dev/null || true
dn: cn=config
changetype: modify
add: olcServerID
olcServerID: 2
EOF
fi

if ldapsearch -H ldapi:/// -Y EXTERNAL -b olcDatabase={1}mdb,cn=config -s base -LLL olcSyncrepl 2>/dev/null | gr
  echo "[init_replication_consumer] olcSyncrepl déjà configuré."
  exit 0
fi

echo "[init_replication_consumer] Configuration olcSyncrepl + olcReadOnly..."
ldapmodify -H ldapi:/// -Y EXTERNAL <<EOF
dn: olcDatabase={1}mdb,cn=config
changetype: modify
add: olcReadOnly
olcReadOnly: TRUE
-
add: olcSyncrepl
olcSyncrepl: rid=001 provider=${PROVIDER_URI} bindmethod=simple binddn="cn=admin,${BASE_DN}" credentials=${ADMIN
EOF

echo "[init_replication_consumer] Terminé."
```

A.9 Script `scripts/init_meta_annuaire.sh`

Dans l'image Docker : copié en `/container/init_meta_annuaire.sh` (hors `init.d`), appelé par `init_ldap.sh` si `LDAP_SERVICE_ROLE=meta`.

```
#!/usr/bin/env bash
# Méta-annuaire OpenLDAP (back-meta) : agrège plusieurs annuaires via suffixmassage.
# Invoqué depuis init_ldap.sh lorsque LDAP_SERVICE_ROLE=meta.
set -euo pipefail
```

```
ADMIN_PASS=${LDAP_ADMIN_PASSWORD:-admin}
META_SUFFIX=${LDAP_META_SUFFIX:-o=federation}
META_ROOT_DN="cn=admin,${META_SUFFIX}"

EX_URI=${LDAP_META_EXAMPLE_URI:-ldap://ldap:389}
EX_REAL=${LDAP_META_EXAMPLE_SUFFIX:-dc=example,dc=org}
EX_VIRT=${LDAP_META_EXAMPLE_VIRTUAL:-ou=example-org,o=federation}

AC_URI=${LDAP_META_ACME_URI:-ldap://ldap-acme:389}
AC_REAL=${LDAP_META_ACME_SUFFIX:-dc=acme,dc=com}
AC_VIRT=${LDAP_META_ACME_VIRTUAL:-ou=acme-corp,o=federation}

export LDAPTLS_REQCERT=never

for _ in $(seq 1 30); do
  if ldapwhoami -H ldapi:/// -Y EXTERNAL >/dev/null 2>&1; then
    break
  fi
  sleep 1
done

echo "[init_meta] Suppression de la base mdb par défaut..."
ldapmodify -H ldapi:/// -Y EXTERNAL <<'EOF' || true
dn: olcDatabase={1}mdb,cn=config
changetype: delete
EOF

echo "[init_meta] Purge des fichiers mdb et redémarrage de slapd..."
pkill -u openldap -TERM slapd 2>/dev/null || pkill -TERM slapd 2>/dev/null || true
for _ in $(seq 1 40); do
  pgrep slapd >/dev/null 2>&1 || break
  sleep 0.25
done
pkill -u openldap -KILL slapd 2>/dev/null || pkill -KILL slapd 2>/dev/null || true
find /var/lib/ldap -mindepth 1 -delete 2>/dev/null || true
chown openldap:openldap /var/lib/ldap
slapd -h "ldap:/// ldapi:///" -u openldap -g openldap -F /etc/ldap/slapd.d &
for _ in $(seq 1 60); do
  if ldapwhoami -H ldapi:/// -Y EXTERNAL >/dev/null 2>&1; then
    break
  fi
  sleep 1
done

if ! ldapsearch -H ldapi:/// -Y EXTERNAL -b cn=module{0},cn=config -s base -LLL olcModuleLoad 2>/dev/null | grep
  echo "[init_meta] Chargement des modules back_ldap (requis par back_meta) puis back_meta..."
  ldapmodify -H ldapi:/// -Y EXTERNAL <<'EOF' || true
dn: cn=module{0},cn=config
changetype: modify
add: olcModuleLoad
olcModuleLoad: back_ldap.la
-
add: olcModuleLoad
olcModuleLoad: back_meta.la
EOF
fi

HASH=$(slappasswd -s "$ADMIN_PASS")
echo "[init_meta] Création de la base meta (suffix=$META_SUFFIX)..."
ldapadd -H ldapi:/// -Y EXTERNAL <<EOF
dn: olcDatabase=meta,cn=config
objectClass: olcDatabaseConfig
```

```
objectClass: olcMetaConfig
olcDatabase: meta
olcSuffix: $META_SUFFIX
olcRootDN: $META_ROOT_DN
olcRootPW: $HASH
olcDbOnErr: continue
olcDbCancel: abandon
olcDbTFSupport: no
olcAccess: to * by dn.exact=$META_ROOT_DN manage by * read
EOF

META_DB_DN=$(ldapsearch -H ldapi:/// -Y EXTERNAL -b cn=config -s one -LLL '(olcDatabase=meta)' dn 2>/dev/null |
if [ -z "$META_DB_DN" ]; then
  echo "[init_meta] ERREUR : entrée olcDatabase meta introuvable dans cn=config."
  exit 1
fi

# Le namingContext dans olcDbURI doit être un sous-arbre du suffixe meta (olcSuffix), pas le suffixe réel du bac
echo "[init_meta] Cible meta : $EX_VIRT → $EX_URI + suffixmassage → $EX_REAL"
ldapadd -H ldapi:/// -Y EXTERNAL <<EOF
dn: olcMetaSub={0}example,$META_DB_DN
objectClass: olcConfig
objectClass: olcMetaTargetConfig
olcMetaSub: {0}example
olcDbURI: $EX_URI/$EX_VIRT
olcDbRewrite: {0}suffixmassage "$EX_VIRT" "$EX_REAL"
olcDbIDAssertBind: bindmethod=simple binddn="cn=admin,$EX_REAL" credentials=$ADMIN_PASS
olcDbChaseReferrals: FALSE
EOF

echo "[init_meta] Cible meta : $AC_VIRT → $AC_URI + suffixmassage → $AC_REAL"
ldapadd -H ldapi:/// -Y EXTERNAL <<EOF
dn: olcMetaSub={1}acme,$META_DB_DN
objectClass: olcConfig
objectClass: olcMetaTargetConfig
olcMetaSub: {1}acme
olcDbURI: $AC_URI/$AC_VIRT
olcDbRewrite: {0}suffixmassage "$AC_VIRT" "$AC_REAL"
olcDbIDAssertBind: bindmethod=simple binddn="cn=admin,$AC_REAL" credentials=$ADMIN_PASS
olcDbChaseReferrals: FALSE
EOF

echo "[init_meta] Méta-annuaire configuré ($META_SUFFIX)."
exit 0
```

A.10 Script `scripts/configure_keycloak_ldap.sh` (machine hôte)

Hors image : à exécuter sur la machine hôte après `docker compose up -d`. Fichier dans le dépôt : `projet/scripts/configure_keycloak_ldap.sh` (ex. `bash projet/scripts/configure_keycloak_ldap.sh` depuis la racine du clone, ou `bash scripts/configure_keycloak_ldap.sh` depuis `projet/`). Pas de second listing dans le corps du guide.

```
#!/usr/bin/env bash
# Configure Keycloak : realm + User Federation LDAP (API d'administration).
# À lancer depuis la machine hôte une fois « docker compose up -d » (Keycloak joindra ldap:389).
set -euo pipefail

KEYCLOAK_URL="${KEYCLOAK_URL:-http://localhost:8090}"
KEYCLOAK_ADMIN="${KEYCLOAK_ADMIN:-admin}"
KEYCLOAK_ADMIN_PASSWORD="${KEYCLOAK_ADMIN_PASSWORD:-admin}"
REALM="${KEYCLOAK_REALM:-tp-ldap}"
LDAP_CONNECTION_URL="${KEYCLOAK_LDAP_CONNECTION:-ldap://ldap:389}"
```

```
LDAP_BIND_DN="${LDAP_BIND_DN:-cn=admin,dc=example,dc=org}"
LDAP_BIND_PW="${LDAP_BIND_PW:-admin}"
LDAP_USERS_DN="${LDAP_USERS_DN:-ou=people,dc=example,dc=org}"

echo "[configure_keycloak_ldap] Attente de Keycloak (${KEYCLOAK_URL})..."
ready=0
for _ in $(seq 1 90); do
  if curl -sf "${KEYCLOAK_URL}/realms/master" >/dev/null; then
    ready=1
    break
  fi
  sleep 2
done
if [ "$ready" != 1 ]; then
  echo "[configure_keycloak_ldap] ERREUR : Keycloak ne répond pas à temps."
  exit 1
fi

echo "[configure_keycloak_ldap] Obtention du jeton admin-cli..."
TOKEN_JSON=$(curl -sS -X POST "${KEYCLOAK_URL}/realms/master/protocol/openid-connect/token" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  --data-urlencode "grant_type=password" \
  --data-urlencode "client_id=admin-cli" \
  --data-urlencode "username=${KEYCLOAK_ADMIN}" \
  --data-urlencode "password=${KEYCLOAK_ADMIN_PASSWORD}")

TOKEN=$(printf '%s' "$TOKEN_JSON" | python3 -c "import sys, json; print(json.load(sys.stdin).get('access_token',
if [ -z "$TOKEN" ]; then
  echo "[configure_keycloak_ldap] ERREUR : impossible d'obtenir access_token. Réponse :"
  printf '%s\n' "$TOKEN_JSON"
  exit 1
fi

AUTH_HDR=(curl -sS -H "Authorization: Bearer ${TOKEN}")
REALM_CODE=$("${AUTH_HDR[@]}" -o /dev/null -w "%{http_code}" "${KEYCLOAK_URL}/admin/realms/${REALM}")

if [ "$REALM_CODE" != "200" ]; then
  echo "[configure_keycloak_ldap] Création du realm « ${REALM} » (HTTP realm=${REALM_CODE})..."
  HTTP_CREATE=$("${AUTH_HDR[@]}" -o /dev/null -w "%{http_code}" -X POST "${KEYCLOAK_URL}/admin/realms" \
    -H "Content-Type: application/json" \
    -d "$(python3 -c "import json; print(json.dumps({'realm':'${REALM}','enabled':True,'displayName':'TP LDAP','
  if [ "$HTTP_CREATE" != "201" ]; then
    echo "[configure_keycloak_ldap] ERREUR : création realm HTTP ${HTTP_CREATE}"
    exit 1
  fi
else
  echo "[configure_keycloak_ldap] Realm « ${REALM} » déjà présent."
fi

EXISTING_ID=$("${AUTH_HDR[@]}" "${KEYCLOAK_URL}/admin/realms/${REALM}/components?type=org.keycloak.storage.UserS
import sys, json
try:
    for c in json.load(sys.stdin):
        if c.get('providerId') == 'ldap':
            print(c.get('id', '') or '')
            break
except json.JSONDecodeError:
    pass
")

if [ -n "$EXISTING_ID" ]; then
  echo "[configure_keycloak_ldap] Fournisseur LDAP déjà configuré (id=${EXISTING_ID})."
  STORAGE_ID="$EXISTING_ID"
```

```
else
  echo "[configure_keycloak_ldap] Création du composant User Federation LDAP..."
  REALM_INTERNAL_ID=$("${AUTH_HDR[@]}" "${KEYCLOAK_URL}/admin/realms/${REALM}" | python3 -c "import sys, json; p

  export _KC_REALM_INTERNAL_ID="$REALM_INTERNAL_ID"
  export _KC_LDAP_URL="$LDAP_CONNECTION_URL"
  export _KC_LDAP_BIND_DN="$LDAP_BIND_DN"
  export _KC_LDAP_BIND_PW="$LDAP_BIND_PW"
  export _KC_LDAP_USERS_DN="$LDAP_USERS_DN"
  BODY=$(python3 <<'PY'
import json, os
print(json.dumps({
  "name": "ldap",
  "providerId": "ldap",
  "providerType": "org.keycloak.storage.UserStorageProvider",
  "parentId": os.environ["_KC_REALM_INTERNAL_ID"],
  "config": {
    "enabled": ["true"],
    "priority": ["0"],
    "importEnabled": ["true"],
    "editMode": ["READ_ONLY"],
    "syncRegistrations": ["false"],
    "vendor": ["other"],
    "usernameLDAPAttribute": ["uid"],
    "rdnLDAPAttribute": ["uid"],
    "uuidLDAPAttribute": ["entryUUID"],
    "userObjectClasses": ["inetOrgPerson, organizationalPerson"],
    "connectionUrl": [os.environ["_KC_LDAP_URL"]],
    "bindDn": [os.environ["_KC_LDAP_BIND_DN"]],
    "bindCredential": [os.environ["_KC_LDAP_BIND_PW"]],
    "usersDn": [os.environ["_KC_LDAP_USERS_DN"]],
    "authType": ["simple"],
    "searchScope": ["2"],
    "pagination": ["true"],
    "connectionPooling": ["true"],
    "startTls": ["false"],
  },
}))
PY
)
  unset _KC_REALM_INTERNAL_ID _KC_LDAP_URL _KC_LDAP_BIND_DN _KC_LDAP_BIND_PW _KC_LDAP_USERS_DN
  CREATE_OUT=$(mktemp)
  HTTP_CODE=$(curl -sS -o "$CREATE_OUT" -w "%{http_code}" -X POST \
    "${KEYCLOAK_URL}/admin/realms/${REALM}/components" \
    -H "Authorization: Bearer ${TOKEN}" \
    -H "Content-Type: application/json" \
    -d "$BODY")
  if [ "$HTTP_CODE" != "201" ]; then
    echo "[configure_keycloak_ldap] ERREUR création LDAP provider (HTTP ${HTTP_CODE}) :"
    cat "$CREATE_OUT"
    rm -f "$CREATE_OUT"
    exit 1
  fi
  rm -f "$CREATE_OUT"
  STORAGE_ID=$("${AUTH_HDR[@]}" "${KEYCLOAK_URL}/admin/realms/${REALM}/components?type=org.keycloak.storage.User
import sys, json
for c in json.load(sys.stdin):
    if c.get('providerId') == 'ldap':
        print(c['id'])
        break
")
  if [ -z "$STORAGE_ID" ]; then
    echo "[configure_keycloak_ldap] ERREUR : id du stockage LDAP introuvable après création."
```

```
    exit 1
  fi
fi

echo "[configure_keycloak_ldap] Synchronisation complète des utilisateurs LDAP..."
SYNC_CODE=$(curl -sS -o /dev/null -w "%{http_code}" -X POST \
  "${KEYCLOAK_URL}/admin/realms/${REALM}/user-storage/${STORAGE_ID}/sync?action=triggerFullSync" \
  -H "Authorization: Bearer ${TOKEN}")
if [ "$SYNC_CODE" != "204" ] && [ "$SYNC_CODE" != "200" ]; then
  echo "[configure_keycloak_ldap] AVERTISSEMENT : sync HTTP ${SYNC_CODE} (souvent acceptable si déjà synchronisé
fi

echo "[configure_keycloak_ldap] Terminé. Console : ${KEYCLOAK_URL}/admin/ - realm « ${REALM} »."
```